

The SPO tree on planted regimes: experiments and results

Daily data with market-sized parameters; risk aversion, fees and depth; split-search options; pruning hurdle

22–23 September 2026

In short. We build an artificial market in which one daily feature x decides the regime of an asset: the price drifts up while x is above a threshold d and down while it is below. The tree trades the asset against cash, sees x alone, and decides every h days. Because we planted the threshold, we know the answer and can measure how close the tree comes. Everything is sized on daily markets: 1% volatility a day, a regime drift of 5 to 25% a year, a feature that mean-reverts over weeks, 15 years of training data, 3 basis points of fees.

Results (Section 4, 2,000 fitted trees).

- With a regime drift of $\pm 25\%$ a year the tree finds the threshold at every trading rate, from daily to monthly: its split is on the planted value to within ± 0.25 standard deviations of x in 60 to 95% of the datasets, and it earns 75 to 85% of what the ideal rule earns out of sample.
- With $\pm 13\%$ a year it still finds it in 62 to 82% of the datasets, less precisely, and captures 25 to 50% of the gain. With $\pm 5\%$ a year nothing can be found in 15 years of data, and the tree mostly keeps no split, which is the right answer: the ideal rule itself earns almost nothing there.
- Daily to weekly decisions are equivalent. Monthly decisions lose precision and profit, all the more when the feature is fast (half-life of a week): the regime then changes inside the holding period.
- A defect was found and fixed on the way: a tree that stayed in cash had an undefined Sharpe ratio, which made pruning reject every split whose alternative was cash (Section 6).
- *Limit:* one clean feature. The earlier experiment (Section 5) showed decoys are ignored at a huge drift; whether they stay ignored at these drifts is the next test.
- *Three more experiments* follow, one section each: risk aversion, fees and depth (Section 8); quantile candidates, refinement and soft splits (Section 9); a calibrated pruning hurdle (Section 10). Each opens with its own one-paragraph summary.

1 The model under test

Decision. Two assets, the risky asset and cash. At a decision the portfolio $w = (w_1, w_2)$, $w \geq 0$, $w_1 + w_2 = 1$, is chosen from the current holdings u by the mean–variance problem with proportional costs (Wang & Hasuiké, “DIR”, eq. 8):

$$w^*(\hat{c}; u, \Sigma) = \arg \max_{w \in \Delta} \hat{c}^\top w - f^\top |w - u| - \lambda w^\top \Sigma w, \quad f = (3 \text{ bp}, 0), \quad \lambda = 1, \quad (1)$$

where $\hat{c}$ is a vector of scores (one per asset) supplied by the predictor and Σ the covariance of the returns until the next decision. The problem is solved exactly (enumeration of the KKT cases).

SPO loss. With the realised returns r , the utility of a portfolio is $U(w; r, u, \Sigma) = r^\top w - f^\top |w - u| - \lambda w^\top \Sigma w$ and the *decision regret* of the scores $\hat{c}$ is

$$\ell(\hat{c}) = U(w^*(r; u, \Sigma)) - U(w^*(\hat{c}; u, \Sigma)) \geq 0, \quad (2)$$

the utility lost by deciding on $\hat{c}$ instead of on the returns that happened (the SPO loss of Elmachetoub & Grigas with costs and risk inside). The predictor is trained on this regret, not on a forecast error.

SPO tree. The predictor is a decision tree on x of depth at most 2, each leaf holding one score vector $\hat{c}_\ell = (\hat{c}_\ell, 0)$ (cash has score 0). It is fitted as in SPOT (Elmachetoub, Liang & McNellis), with two additions of this implementation.

- *Growth*, on the first 75% of the training decisions. At each round and for every leaf, every threshold on x is screened, the three best are refitted (leaf scores by a bounded coordinate search on the empirical regret, $|\hat{c}_\ell| \leq 10\%$, at least 20 decisions per leaf), and the split with the largest total regret reduction is accepted if the replayed training Sharpe ratio rises. The holdings u_t in (2) are those of the tree’s own replayed path: fees depend on what was held before.
- *Pruning*, on the last 25% (the “checking” decisions, unseen during growth): a split stays only if, there, it lowers the mean regret and raises the Sharpe ratio. Leaf scores are then refitted on all training decisions.

The optimizer, the tree and its settings are those of `run_real_data.py` (the real-data pipeline).

Two clocks. Data are daily; decisions are taken every h days and held in between. A training row is one decision: x_t on the decision day, the label $r_t = S_{t+h}/S_t - 1$ for the asset (0 for cash) and $\Sigma_t = \text{diag}(h\hat{\sigma}_t^2, 10^{-6})$ with $\hat{\sigma}_t^2$ the variance of the last 180 daily returns. h is the trading rate.

2 The planted world

The feature is a daily Ornstein–Uhlenbeck process and the asset a geometric random walk whose drift follows the feature (`synthetic_market.py`):

$$x_{t+1} = m + e^{-k}(x_t - m) + \eta_t, \quad \eta_t \sim \mathcal{N}(0, s^2(1 - e^{-2k})), \quad (3)$$

$$\log \frac{S_{t+1}}{S_t} = \mu_t - \frac{1}{2}\sigma^2 + \sigma\varepsilon_t, \quad \mu_t = \begin{cases} +\mu & x_t \geq d \\ -\mu & x_t < d \end{cases}, \quad \mu = e\sigma. \quad (4)$$

x has long-run mean m , stationary variance s^2 and half-life $\ln 2/k$ days. The *edge* $e = \mu/\sigma$ is the daily drift-to-volatility ratio, so $e\sqrt{252}$ is the annual Sharpe ratio of a trader who knows the regime every day. The threshold is planted **at the mean of the feature**, $d = m$.

What a correct tree should find. A decision is held for h days while x keeps moving, so what matters at a decision is the expected return over the holding period given x now. With $B_h = \sum_{j=0}^{h-1} \text{sign}(x_j - d)$ the number of days spent above d minus the number below,

$$g_h(x) = \mathbb{E}[e^{\mu B_h} \mid x_0 = x] - 1 \quad (5)$$

(the price noise averages out), computed by Monte Carlo on (3). Because $d = m$, the law of $x - m$ is symmetric and $g_h(m + z) = -g_h(m - z)$ up to the convexity of the exponential (negligible here): **g_h changes sign at d for every holding period.** By (1) the ideal position is the full asset where $g_h(x) > 2\lambda h\sigma^2$ (0.02% for $h = 1$, 0.42% for $h = 21$) and cash below, so the decision changes at d up to a few hundredths. The right threshold is therefore d at every trading rate. (When $d \neq m$ this is no longer true: see Section 5.)

Parameter	Base	Also tested	Meaning and realism
σ	1% a day	–	volatility of the asset, 16% a year (an equity index)
$e = \mu/\sigma$	0.10	0.02, 0.05	drift $\pm 25\%$ a year ($\pm 5\%$, $\pm 13\%$); Sharpe of a perfect daily timer $e\sqrt{252} = 1.6$ (0.3, 0.8)
k	0.02 a day	0.01, 0.05, 0.10	reversion speed: half-life 35 days (69, 14, 7 days)
m, s^2, d	0, 1, 0	–	x is a standardized feature; the threshold sits at its mean
h	5 days	1, 2, 10, 21	trading rate: weekly (daily, twice a week, every two weeks, monthly)
training, test	15 years, 5 years	–	3,780 and 1,260 daily observations; 180 warm-up days for Σ
fee, λ	3 bp, 1	–	per traded notional on the asset, 0 on cash
tree	depth ≤ 2 , ≥ 20 per leaf	–	all thresholds screened, shortlist 3, last 25% of the training decisions prune
datasets per cell	100	50	100 at $e \in \{0.05, 0.10\}$, 50 at $e = 0.02$ and in the k sweep

Table 1: Parameters (the defaults of `Case` in `planted_threshold_test.py` and the sweeps of `planted_threshold_experiments.py`). One decision every h days over 15 years gives 3,780, 1,890, 756, 378 and 180 training decisions for $h = 1, 2, 5, 10, 21$ (the last quarter for pruning), and 1,260, 630, 252, 126, 60 test decisions.

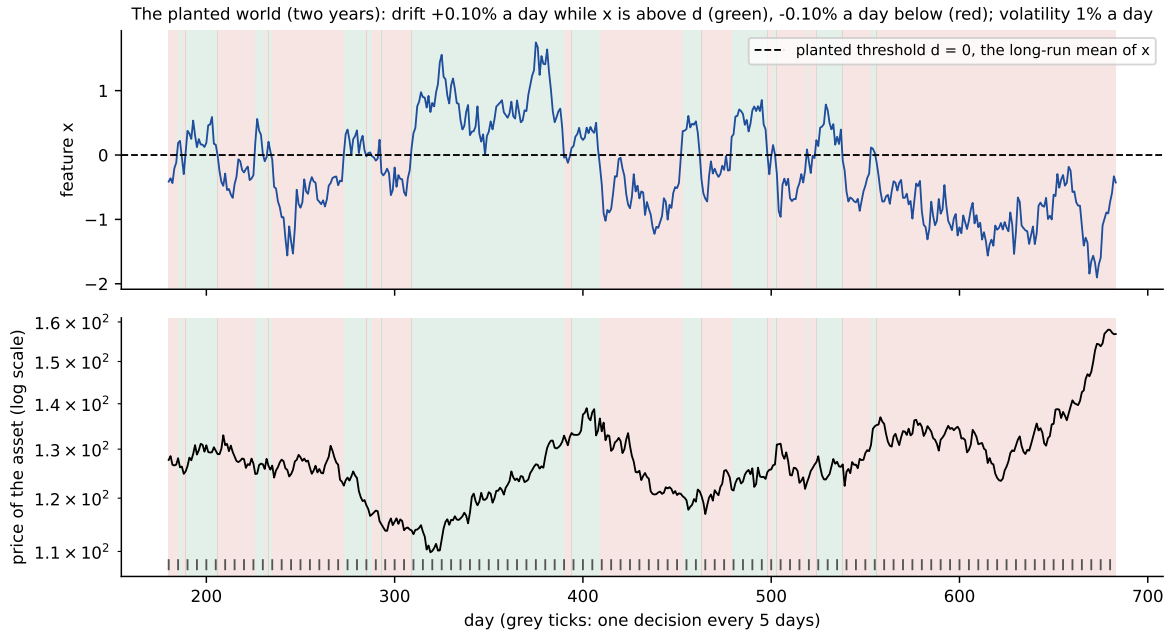


Figure 1: Two years of one dataset (base parameters, weekly decisions). The background is the true regime, which the tree never sees; a decision is taken at each grey tick and held until the next one.

3 Protocol

Pipeline used. P2 of `model_history.md`: the tree of `model_and_training_pipeline.tex` with every option off (plain splits, every threshold screened, depth 2, 20 decisions per leaf, pruning on the last quarter of the training decisions, $\lambda = 1$, fee 3 bp), with the Sharpe ratio of an all-cash path defined as 0; market M2 (Section 2).

For each cell (a trading rate h with an edge e , or with a speed k at $e = 0.10$) and each of its N independent datasets: simulate $180 + 3,780 + 1,260$ days; fit the tree on the 15 training years;

replay it on the 5 test years, starting from its holdings at the end of training. Four benchmarks are replayed on the same rows with the same optimizer: the *ideal rule* (optimizer fed with $\hat{c} = (g_h(x), 0)$, the best possible use of x); the *rule at d* (scores $\pm 10\%$ according to $x \geq d$); *always the asset*; the *tree without splits* (one leaf, i.e. no information). Four measurements:

1. **Does it split, and where?** Share of datasets whose first split is on x (with one feature, the alternative is no split); the distribution of the first threshold, whose target is $d = 0$ in units of the standard deviation of x .
2. **Test Sharpe ratio**, annualized as $\sqrt{252/h} \times$ the per-decision Sharpe of net returns, averaged over the datasets.
3. **Share of the possible gain captured:** $(R_{\text{tree}} - R_{\text{flat}})/(R_{\text{ideal}} - R_{\text{flat}})$ with R the mean test return, pooled over the datasets of a cell (blank when the ideal rule gains under 0.5% a year: nothing to capture).
4. **No split kept:** share of datasets where pruning left a single leaf.

4 Results

4.1 One dataset

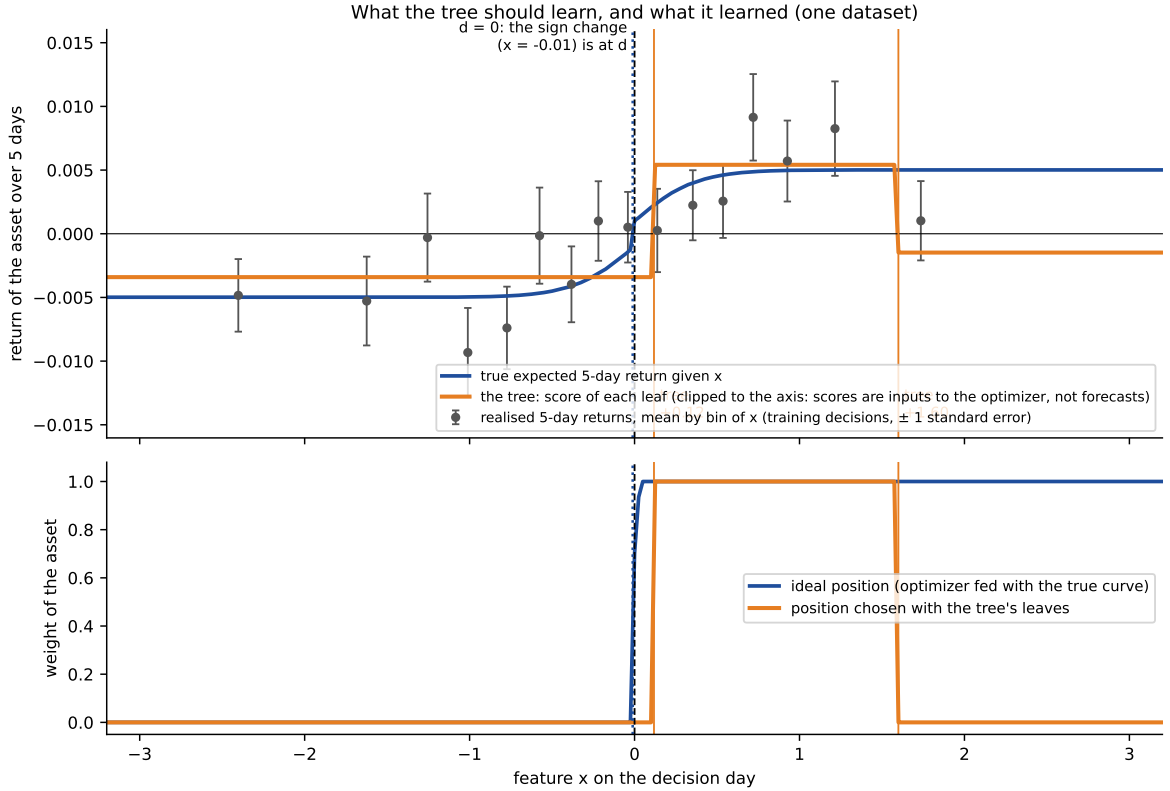


Figure 2: Base parameters, weekly decisions, dataset 42. *Top:* realised 5-day returns by bin of x (training decisions), the true $g_5(x)$ of (5), and the tree's leaf scores. *Bottom:* the position the optimizer takes with the true curve and with the tree. Growth made three splits. Pruning removed the one at $x \leq -2.01$ and kept the root split at $x \leq 0.12$ (on the checking decisions: regret $0.0097 \rightarrow 0.0066$ per decision, Sharpe $0.06 \rightarrow 0.20$) together with a second-level one at $x \leq 1.60$ (31 decisions, regret $0.0067 \rightarrow 0.0066$), an example of the tail leaves discussed in Section 7.

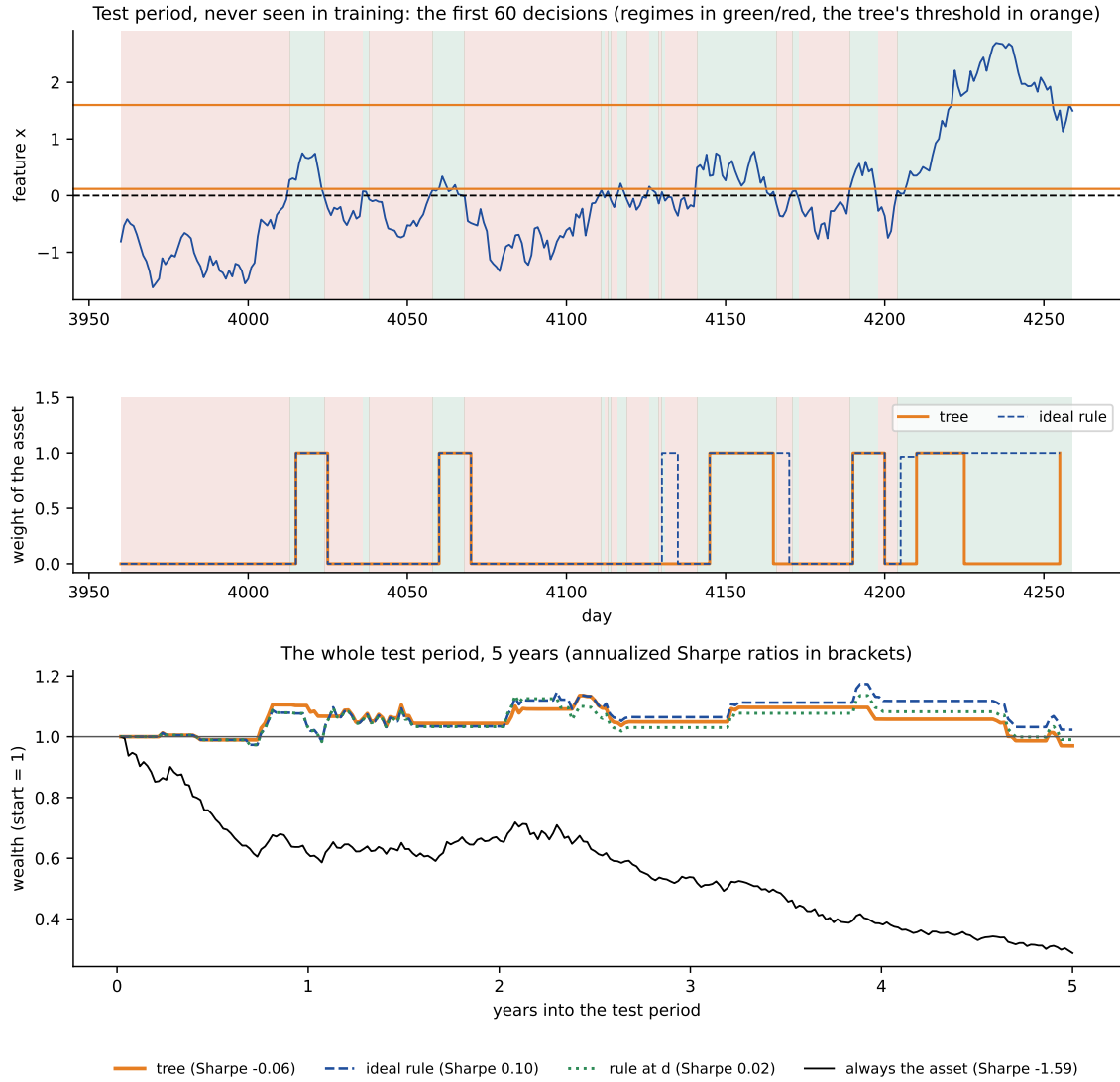


Figure 3: The same dataset out of sample. *Top, middle:* the first 60 test decisions; the tree is in cash below its threshold and fully invested above, like the ideal rule. *Bottom:* wealth over the 5 test years. This test window is dominated by the down regime (the asset loses 22% a year), so every rule earns little; what the figure shows is the decisions, not the level of the returns.

4.2 Where the threshold lands, over 100 datasets per cell

Where the tree puts its thresholds on x ($k = 0.02$, 100 datasets per panel; x has standard deviation 1)

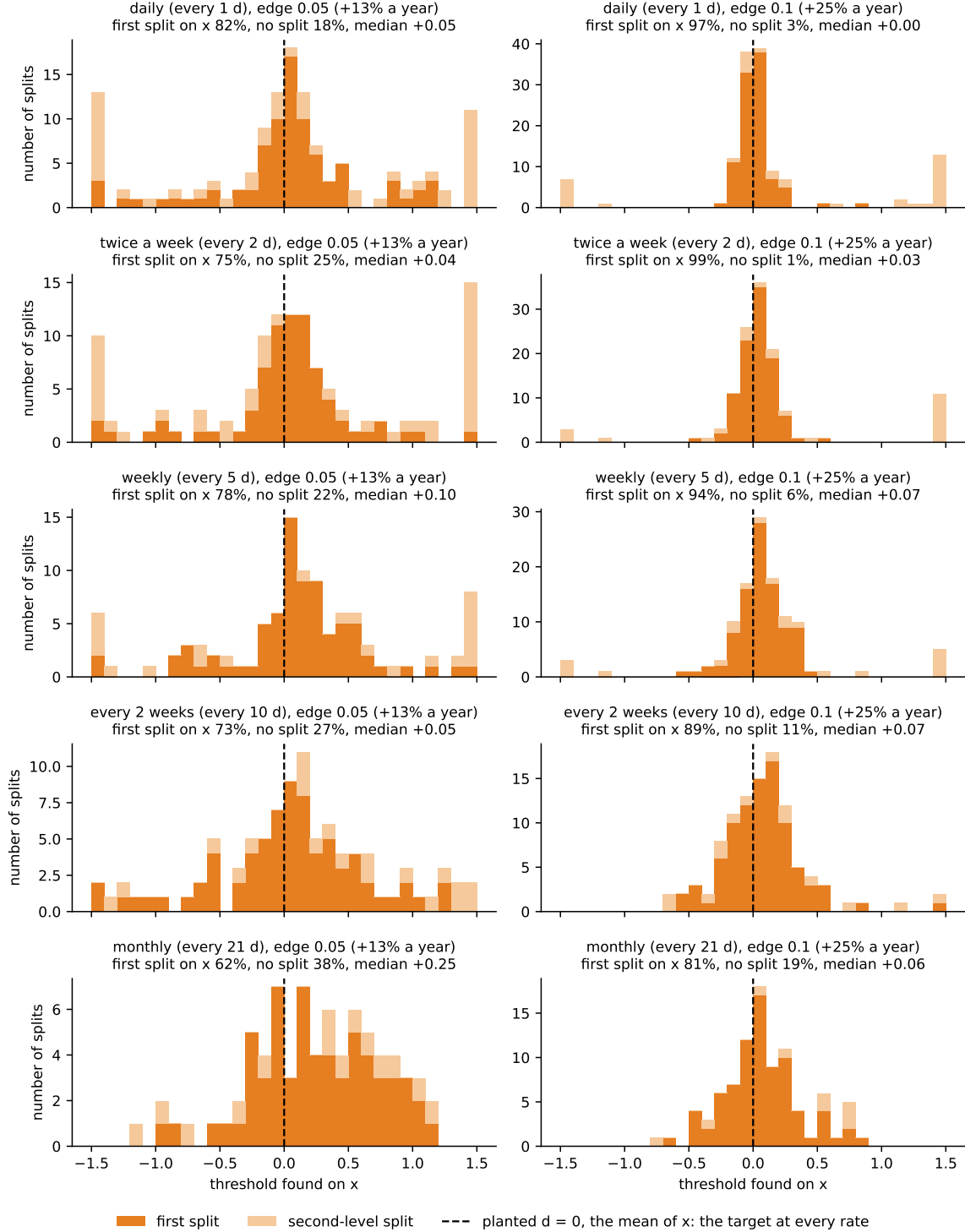


Figure 4: All thresholds on x kept in the final trees, one panel per trading rate, $k = 0.02$; *top*: edge 0.05, *bottom*: edge 0.10. The target is $d = 0$ at every rate; x has standard deviation 1. Bars at ± 1.5 collect thresholds beyond the axis (tail leaves).

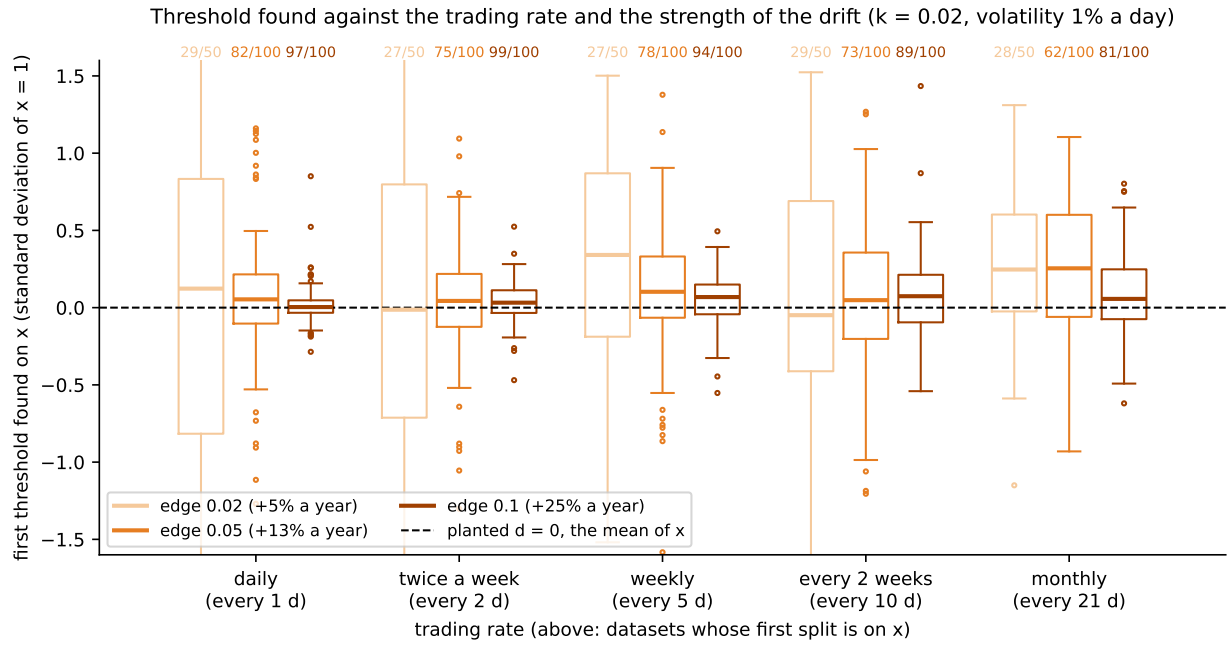


Figure 5: First threshold found, by trading rate and edge (boxes: quartiles; the count above each box is the number of datasets that split).

4.3 Trading rate against edge, and against the speed of the feature

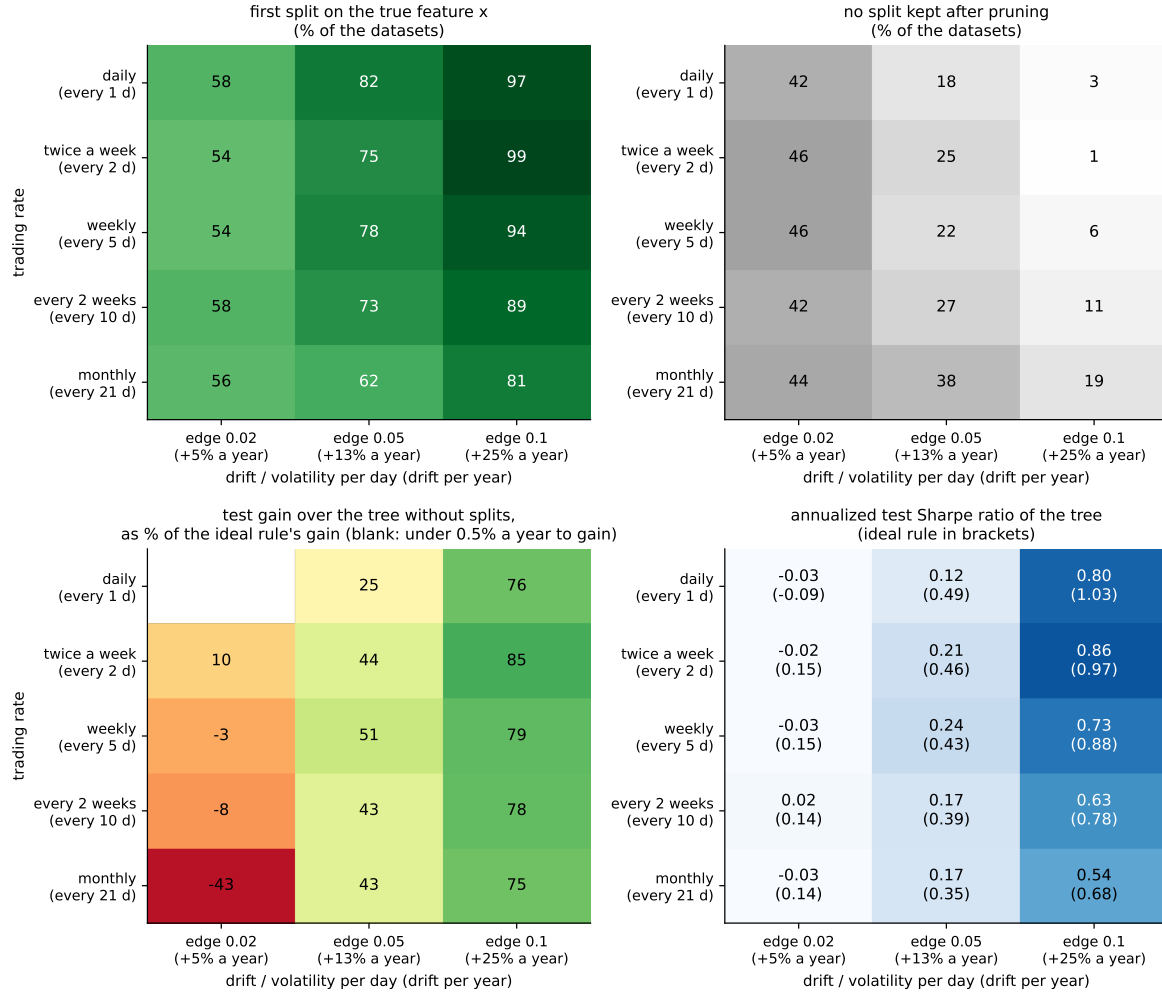


Figure 6: Rows: trading rate; columns: edge ($k = 0.02$). From left to right: datasets whose first split is on x ; datasets that keep no split; share of the ideal rule's test gain captured; annualized test Sharpe ratio of the tree (ideal rule in brackets).

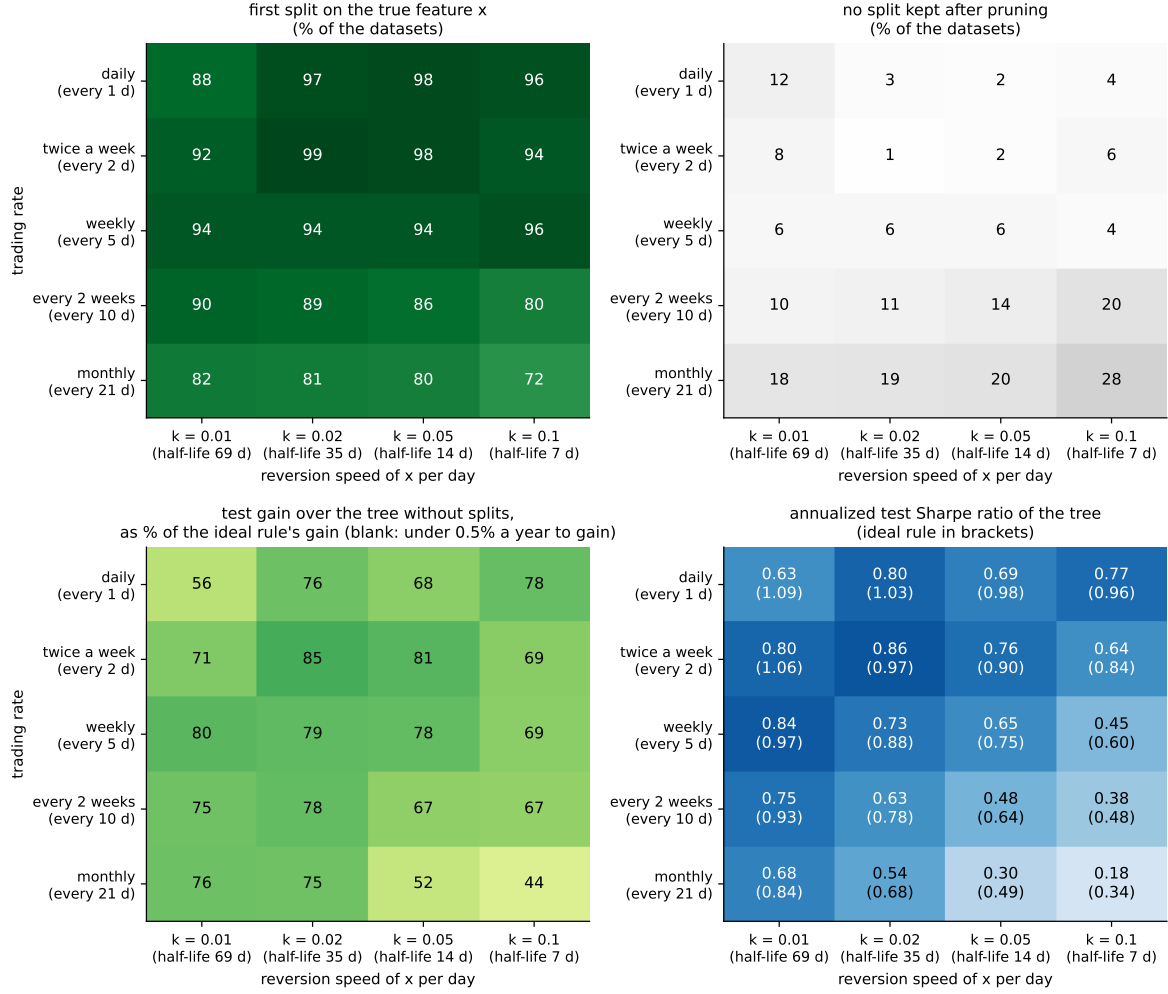


Figure 7: Same maps with the reversion speed of x in columns, at edge 0.10.

edge	h	splits (%)		first threshold on x			annualized test Sharpe			gain
		on x	none	median [quartiles]	$ \cdot \leq 0.25$		tree	ideal	at d	captured
0.10	1	97	3	+0.00 [−0.03, +0.05]	95%		0.80	1.03	1.04	76%
	2	99	1	+0.03 [−0.03, +0.11]	92%		0.86	0.97	0.98	85%
	5	94	6	+0.07 [−0.04, +0.15]	81%		0.73	0.88	0.88	79%
	10	89	11	+0.07 [−0.10, +0.21]	70%		0.63	0.78	0.78	78%
	21	81	19	+0.06 [−0.07, +0.25]	60%		0.54	0.68	0.66	75%
0.05	1	82	18	+0.05 [−0.10, +0.22]	60%		0.12	0.49	0.49	25%
	2	75	25	+0.04 [−0.12, +0.22]	64%		0.21	0.46	0.46	44%
	5	78	22	+0.10 [−0.07, +0.33]	53%		0.24	0.43	0.42	51%
	10	73	27	+0.05 [−0.20, +0.36]	47%		0.17	0.39	0.38	43%
	21	62	38	+0.25 [−0.06, +0.60]	34%		0.17	0.35	0.34	43%
0.02	1	58	42	+0.12 [−0.82, +0.83]	24%	−0.03	−0.09	0.14		—
	2	54	46	−0.01 [−0.71, +0.80]	15%	−0.02	0.15	0.13		10%
	5	54	46	+0.34 [−0.19, +0.87]	26%	−0.03	0.15	0.13		−3%
	10	58	42	−0.05 [−0.41, +0.69]	24%	0.02	0.14	0.13		−8%
	21	56	44	+0.25 [−0.02, +0.60]	32%	−0.03	0.14	0.14		−43%

Table 2: Trading rate h (days between decisions) against edge, $k = 0.02$; 100 datasets per row at edges 0.10 and 0.05, 50 at 0.02. “Always the asset” has an annualized Sharpe between -0.04 and 0.00 everywhere (no unconditional drift). Standard error of the tree–ideal Sharpe difference: 0.03 to 0.05.

half-life of x	h	splits (%)		first threshold on x		annualized Sharpe		gain
		on x	none	median [quartiles]	$ \cdot \leq 0.25$	tree	ideal	captured
69 days ($k = 0.01$)	1	88	12	+0.00 [−0.03, +0.08]	86%	0.63	1.09	56%
	2	92	8	+0.05 [−0.00, +0.08]	89%	0.80	1.06	71%
	5	94	6	+0.05 [−0.04, +0.12]	87%	0.84	0.97	80%
	10	90	10	+0.06 [−0.11, +0.14]	87%	0.75	0.93	75%
	21	82	18	+0.15 [−0.06, +0.23]	71%	0.68	0.84	76%
14 days ($k = 0.05$)	1	98	2	+0.00 [−0.01, +0.06]	94%	0.69	0.98	68%
	2	98	2	+0.05 [−0.01, +0.15]	88%	0.76	0.90	81%
	5	94	6	+0.18 [−0.02, +0.25]	74%	0.65	0.75	78%
	10	86	14	+0.21 [−0.05, +0.43]	44%	0.48	0.64	67%
	21	80	20	+0.12 [−0.16, +0.34]	40%	0.30	0.49	52%
7 days ($k = 0.10$)	1	96	4	+0.02 [−0.02, +0.07]	92%	0.77	0.96	78%
	2	94	6	+0.03 [−0.02, +0.16]	74%	0.64	0.84	69%
	5	96	4	+0.17 [+0.04, +0.45]	52%	0.45	0.60	69%
	10	80	20	+0.20 [−0.14, +0.54]	38%	0.38	0.48	67%
	21	72	28	+0.10 [−0.03, +0.58]	47%	0.18	0.34	44%

Table 3: Trading rate against the speed of x , at edge 0.10, 50 datasets per row (the 35-day row is the edge-0.10 block of Table 2).

Reading the figures and tables.

- At edge 0.10 the tree splits on x in 81 to 99% of the datasets and puts its threshold at the mean: median 0.00 to 0.07, quartiles within ± 0.05 daily and $[-0.07, +0.25]$ monthly. Out of sample it is 0.11 to 0.23 of annual Sharpe below the ideal rule (standard error 0.03–0.04) and captures 75 to 85% of the gain. The rule at d equals the ideal rule: with these sizes the ideal position is all or nothing.
- At edge 0.05 the threshold is found in 62 to 82% of the datasets but with quartiles of about ± 0.2 , and 25 to 51% of the gain is captured. At edge 0.02 the thresholds spread over the whole range and no gain is captured; the ideal rule itself reaches a Sharpe of 0.15 at most. Fifteen years of data do not contain that signal.
- The trading rate matters little from daily to weekly. Every two weeks and monthly lose ground (81% found and 60% within ± 0.25 at $h = 21$ against 97% and 95% daily), for two reasons visible in Table 3: fewer decisions to learn from (180 monthly ones) and, when x is fast, a regime that flips inside the holding period ($g_{21}(1) = 0.87\%$ at $k = 0.10$ against 1.79% at $k = 0.02$).
- The slow rates show a small upward bias of the median threshold (+0.06 to +0.25): with a holding period and fees, entering just above the mean is not worth the trade, so the tree waits for x to be clearly positive.
- The no-split share rises from 3% (daily) to 19% (monthly) at edge 0.10 and reaches 42–46% at edge 0.02: when the checking decisions do not confirm a split, the tree stays flat, which is the safe side.

5 The first experiment (18 September): a large drift with decoys

Pipeline used. P1: the same tree before the all-cash Sharpe convention (an all-cash path had an undefined Sharpe ratio); market M1, with three decoy features next to x .

The first run used the same tree, optimizer and price model, but with a step as the time unit and sizes far from markets: $\mu = 3\%$ per step for $\sigma = 3.2\%$ per step, an edge of about 0.95 instead of 0.02–0.10. It also placed the threshold *below* the mean of x and hid x among decoys. It is kept here

because it answers two questions the new run does not: are decoys ignored, and what does the tree find when $d \neq m$?

Parameter	Value	Note
μ, σ	3%, 3.2% per step	edge 0.95; volatilities 1 to 30% per step also swept
$d; m, s^2$	0; 1, 4	d half a standard deviation below the mean: $x \geq d$ 69% of the time
k	0.2 per step	a regime lasts about 5 steps; 0.01 to 0.5 also swept
h	10 steps (and 1)	a position held while the regime changes twice on average
features	x , 20-step average of x , an unrelated OU with the same law, white noise	three decoys
data	443 training decisions (332 grow, 111 prune), 300 test	100 datasets per histogram, 20 per cell of the $k \times \sigma$ map, 800 fits

Table 4: The first experiment.

What should the tree find when $d \neq m$? Not d . Held for 10 steps, a position taken just below d is soon above it, because x reverts to $m = 1 > d$. The expected 10-step return g_{10} of (5) changes sign at $x \approx -0.92$, and the ideal position rises from 0 to 1 between there and about -0.3 . The right question was “does the tree put its threshold in $[-0.9, -0.3]$?”, and d itself is the target only for $h = 1$.

Results.

- $h = 1$ (target d): first split on x in 100 datasets of 100, one split per tree, never a decoy; median threshold 0.000, 95 of 100 within 0.1 of d ($s = 2$). Test Sharpe per decision: tree 0.684, ideal rule 0.682, always the asset 0.270.
- $h = 10$ (target $[-0.9, -0.3]$): first split on x in 86 of 100, never first on a decoy, no split in 14; median -0.46 , 90% between -1.53 and $+0.01$. Test Sharpe per decision: tree 0.724, ideal 0.742, rule at d 0.749, always the asset 0.628 (tree -0.018 ± 0.004 against the ideal rule). Fifteen trees kept a second split on x , three a second-level split on the noise decoy.
- $k \times \sigma$ map ($h = 10$, 600 trees): x is found first in 95–100% of the datasets when a regime lasts at least about one holding period ($k \leq 0.1$) and $\sigma \leq 20\%$; at $k = 0.5$ (a regime of 2 steps) in 20–45%, and then the usual outcome is no split, which is right (the ideal rule gains 0.01% per decision). Over the 600 trees the noise decoy is the first split in 9 (1.5%) and the moving average in 30 (5%), almost all at $\sigma \geq 20\%$.
- The sign change of g_{10} moves with k ($-0.04, -0.05, -0.16, -0.37, -0.92, -3.6$ for $k = 0.01$ to 0.5) and the tree’s median thresholds follow it ($-0.02, +0.02, -0.04, -0.19, -0.47, -1.52$), staying between the sign change and d .

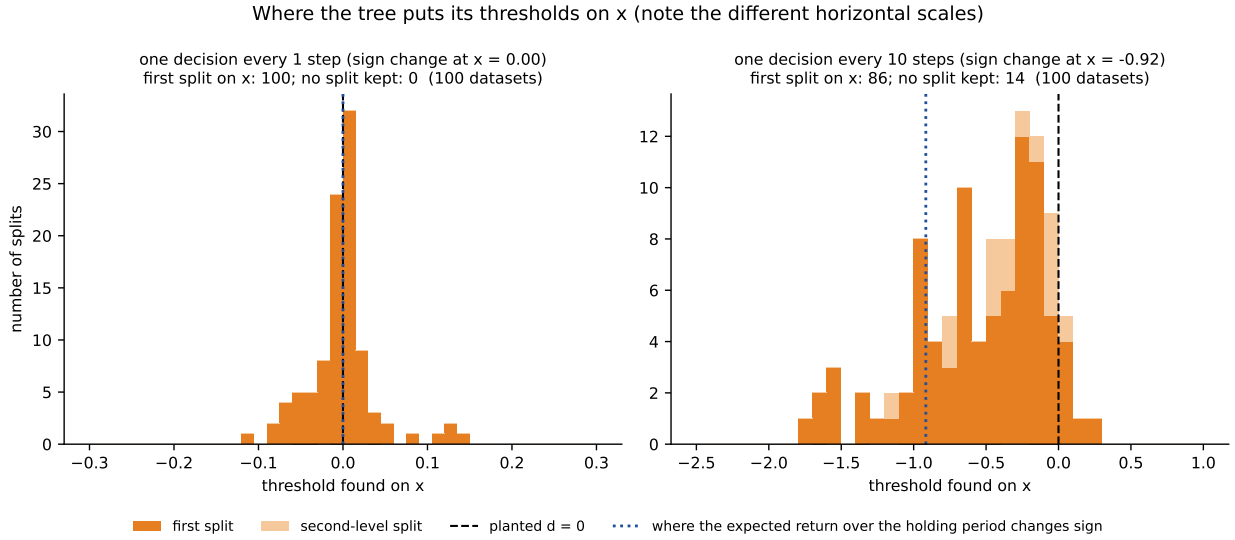


Figure 8: First experiment: thresholds kept, 100 datasets per panel. *Left:* $h = 1$, target d . *Right:* $h = 10$, target between the sign change (dotted) and about -0.3 .

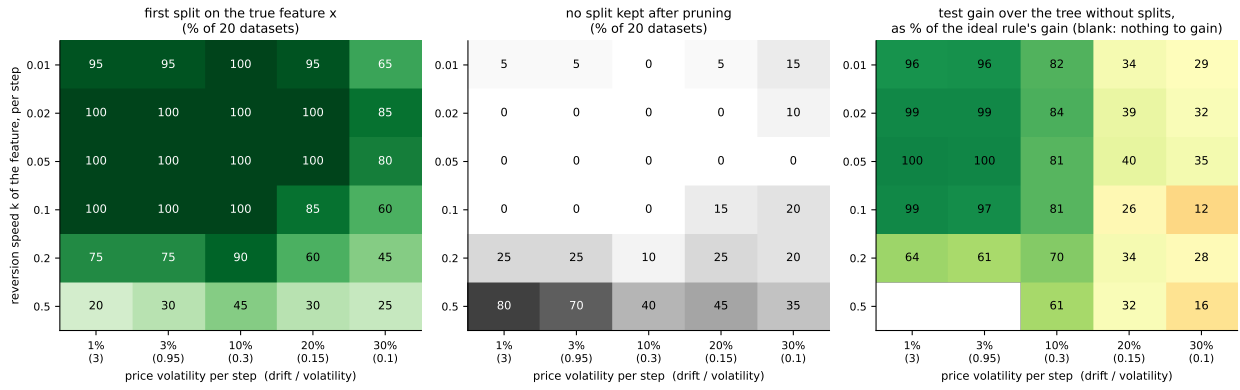


Figure 9: First experiment, $h = 10$, 20 datasets per cell: rows are the speed k of x , columns the volatility of the price (edge in brackets).

Why it was not enough. An edge of 0.95 means the regime is visible in a handful of decisions; the experiments of the main report on real data worked near 0.07. The present run repeats the test at 0.02–0.10, which is what connects it to markets.

6 What changed in the code

- `planted_threshold_test.py`: the time unit is the trading day (252 a year); the tree sees x alone; $d = m$; the training and test spans are given in days (15 and 5 years), so that every trading rate sees the same history; the drift is given as an edge $e = \mu/\sigma$; results are annualized by $252/h$.
- `planted_threshold_experiments.py`: the sweeps are trading rate $\times$ edge and trading rate $\times$ speed; the maps carry the annualized Sharpe ratios; a summary table is printed by `plot`.
- `spo_tree.py`, one definition: the Sharpe ratio of a path with fewer than two periods stays undefined, but a path with constant zero net returns (all cash, no trade) now has a Sharpe ratio of 0 instead of NaN. Every comparison “Sharpe_{with split} > Sharpe_{without}” was false when the tree without the split sat in cash, so pruning removed every such split, however good; with a zero-mean asset this happened in most datasets. A unit test covers it.

7 What this test says, and what it does not

- **The mechanism works at market sizes.** With a regime drift of $\pm 25\%$ a year and a feature that persists for a month, the tree recovers the planted threshold at every trading rate and delivers most of the attainable Sharpe ratio out of sample. At $\pm 13\%$ it degrades gracefully; at $\pm 5\%$ it correctly finds nothing.
- **Deciding more often does not teach more about a slow signal** (daily to weekly are equivalent), but deciding too rarely loses the fast part of the signal and the number of decisions to learn from.
- **Tail leaves.** With 3,780 daily rows the minimum of 20 decisions per leaf is 0.7% of the data, and pruning lets some second-level splits in the tails through (the bars at ± 1.5 of Figure 4); they cost little here but the minimum leaf size should scale with the number of rows.
- **Not shown:** decoys at these edges, an unconditional premium of the asset, several assets, a regime depending on more than one threshold, volatility that changes with the regime.

8 A second experiment: risk aversion, fees, depth and quantile thresholds

Pipeline used. P2 with λ , the fee and the depth varied as listed; the “quartiles” setting used the quantiles of the node’s *values* as candidates, the rule in force on 22 September, not the rank rule of Section 9.

Why. The positions of Section 4 are 0 or 1. With the fee f on the asset and holdings u , the optimizer (1) gives

$$w = \frac{\hat{c} \mp f}{2\lambda\sigma_h^2} \text{ clipped to } [0, 1], \quad \text{and } w = u \text{ inside the no-trade zone } |\hat{c} - 2\lambda\sigma_h^2 u| \leq f, \quad (6)$$

so a position is interior only for scores within a window of width $2\lambda\sigma_h^2$: 0.1% at weekly decisions with $\lambda = 1$, against expected returns of $\pm 0.5\%$. The ideal position is then a step, and a step is what the tree learns. To see whether the tree can learn a *gradual* position, and how fees, depth and the number of candidate thresholds change the picture, the same world was run again with one or two things changed at a time (Table 5): 40 datasets per cell, edges 0.05 and 0.10, $k = 0.02$, decisions every 2, 5, 10 and 21 days (daily decisions were left out: two thirds of the compute for the same result as every 2 days in Section 4). One measurement is new: the mean weight taken on the test decisions, by bin of x (Figure 10).

Setting	What changes with respect to the run of Section 4 (base)
depth 3	up to 8 leaves instead of 4
$\lambda = 5$, depth 3	risk aversion 5: the window of (6) is 0.5% wide at weekly decisions
$\lambda = 10$, depth 3	risk aversion 10: window 1% wide; the ideal position never exceeds one half
fee 10 bp	fee 10 basis points per trade instead of 3
quartiles, depth 3	only three candidate thresholds per split, the quartiles of the node’s values (<code>threshold_quantiles=(0.25,0.5,0.75)</code>); every leaf then holds at least a quarter of its parent

Table 5: The settings experiment. The settings are fields of Case (`risk_aversion`, `fee`, `max_depth`, `threshold_quantiles`); their defaults reproduce Section 4.

Position taken on the test decisions, by x (weekly decisions, 40 datasets per panel)

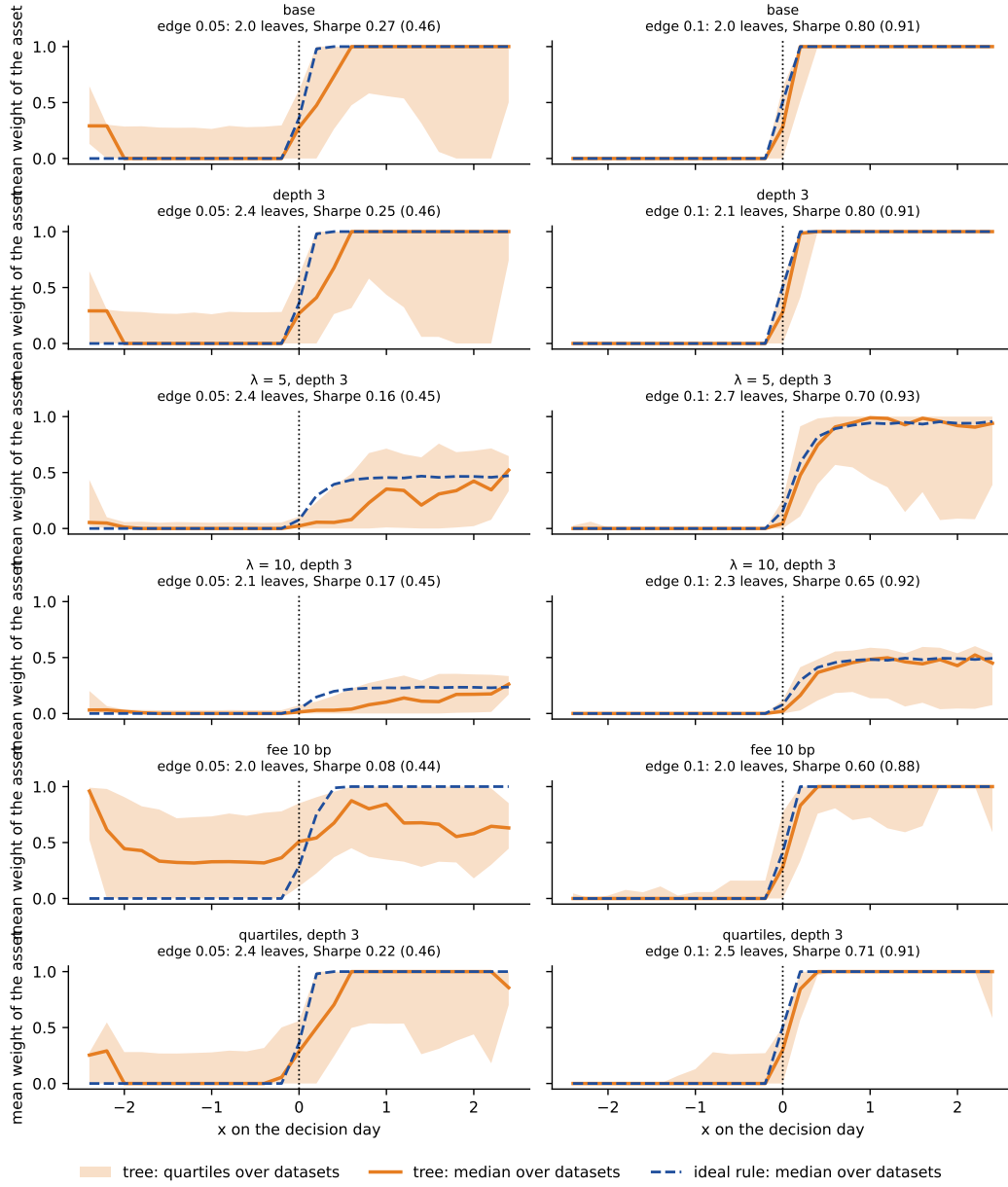


Figure 10: Weekly decisions: the weight of the asset taken on the test decisions, by x on the decision day. Orange: the tree (median over the 40 datasets, and the quartiles as a band); dashed blue: the ideal rule, the optimizer fed with the true $g_5(x)$. One row per setting; *left*: edge 0.05, *right*: edge 0.10. Panel titles: mean number of leaves, annualized test Sharpe ratio of the tree (ideal rule in brackets).

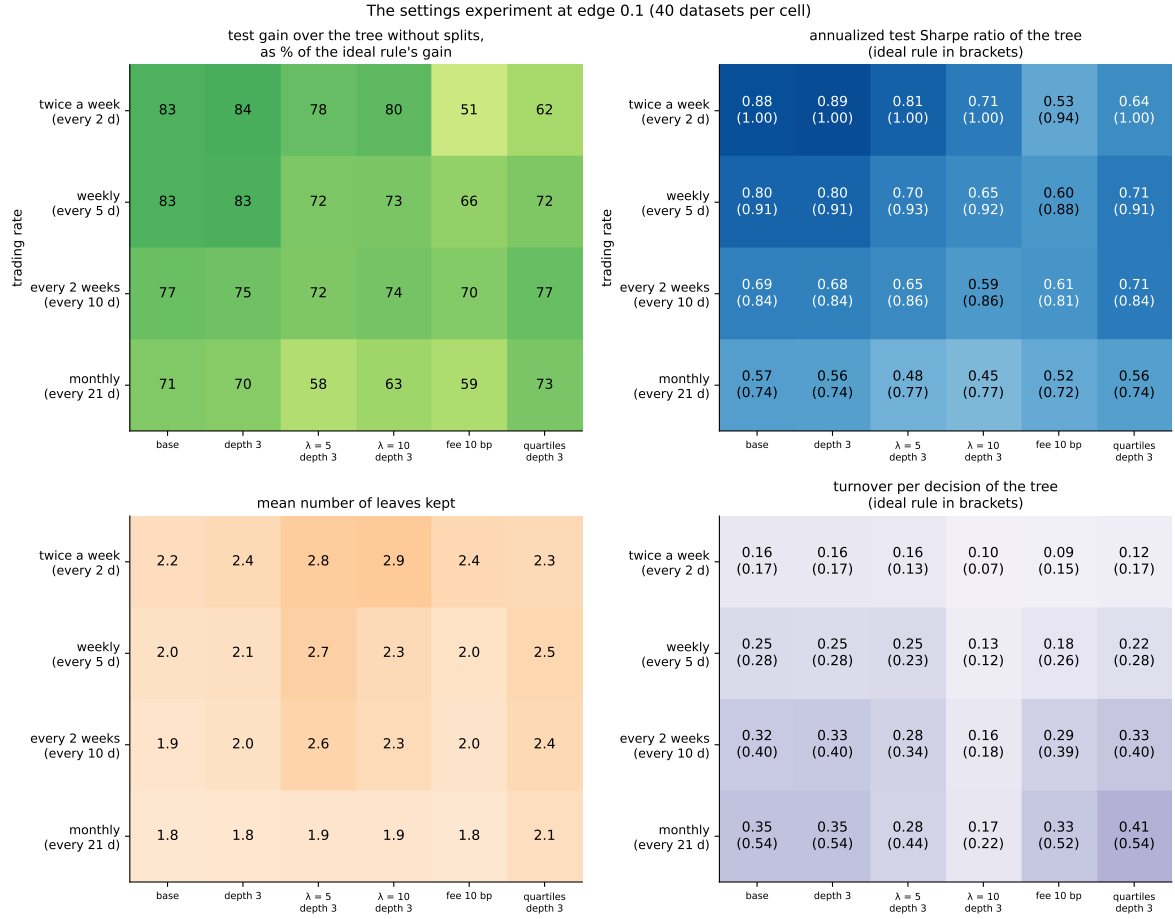


Figure 11: Edge 0.10, rows: trading rate, columns: setting. Share of the ideal rule's test gain captured; annualized test Sharpe ratio (ideal rule in brackets); mean number of leaves kept; turnover per decision (ideal rule in brackets).

setting	h	leaves	splits (%)		threshold median	annual. Sharpe		gain captured	turnover	
			on x	none		tree	ideal		tree	ideal
base	2	2.2	98	2	+0.03	0.88	1.00	83%	0.16	0.17
	5	2.0	92	8	+0.08	0.80	0.91	83%	0.25	0.28
	10	1.9	88	12	+0.05	0.69	0.84	77%	0.32	0.40
	21	1.8	72	28	+0.09	0.57	0.74	71%	0.35	0.54
depth 3	2	2.4	98	2	+0.03	0.89	1.00	84%	0.16	0.17
	5	2.1	92	8	+0.08	0.80	0.91	83%	0.25	0.28
	10	2.0	88	12	+0.05	0.68	0.84	75%	0.33	0.40
	21	1.8	72	28	+0.09	0.56	0.74	70%	0.35	0.54
$\lambda = 5$, depth 3	2	2.8	88	12	+0.12	0.81	1.00	78%	0.16	0.13
	5	2.7	82	18	+0.23	0.70	0.93	72%	0.25	0.23
	10	2.6	80	20	+0.26	0.65	0.86	72%	0.28	0.34
	21	1.9	68	32	+0.43	0.48	0.77	58%	0.28	0.44
$\lambda = 10$, depth 3	2	2.9	85	15	+0.18	0.71	1.00	80%	0.10	0.07
	5	2.3	75	25	+0.25	0.65	0.92	73%	0.13	0.12
	10	2.3	78	22	+0.28	0.59	0.86	74%	0.16	0.18
	21	1.9	70	30	+0.44	0.45	0.77	63%	0.17	0.22
fee 10 bp	2	2.4	95	5	+0.06	0.53	0.94	51%	0.09	0.15
	5	2.0	85	15	+0.08	0.60	0.88	66%	0.18	0.26
	10	2.0	85	15	+0.09	0.61	0.81	70%	0.29	0.39
	21	1.8	68	32	+0.09	0.52	0.72	59%	0.33	0.52
quartiles, depth 3	2	2.3	92	8	+0.06	0.64	1.00	62%	0.12	0.17
	5	2.5	85	15	+0.09	0.71	0.91	72%	0.22	0.28
	10	2.4	92	8	+0.09	0.71	0.84	77%	0.33	0.40
	21	2.1	80	20	+0.06	0.56	0.74	73%	0.41	0.54

Table 6: The settings experiment at edge 0.10, 40 datasets per row (h : days between decisions). “Leaves”: mean number of leaves kept; “median”: first threshold on x over the datasets that split. At edge 0.05 no setting beats the base: the tree’s Sharpe ratio is 0.19–0.27 for the base and 0.12–0.19 with $\lambda = 5$ or 10, and with a 10 bp fee the tree hardly trades at the fast rates (0.08–0.09).

Reading the figures and the table.

- **Depth alone changes nothing:** same leaves, same thresholds, same Sharpe ratios as the base to the second decimal. With $\lambda = 1$ the ideal position is a step and the tree has no reason to use more leaves.
- **With $\lambda = 5$ or 10 the ideal position is a ramp** (to a plateau of one half at $\lambda = 10$) **and the tree follows it**, with 2.3 to 2.9 leaves: it does learn a gradual position when the decision problem asks for one (Figure 10, bottom row). The price is estimation: 72 to 80% of the gain captured against 83%, a Sharpe ratio 0.1 to 0.15 lower, thresholds that move up (+0.12 to +0.44) and more datasets without a split. The ideal rule itself gains nothing from sizing at these signal sizes (its Sharpe ratio is the same at $\lambda = 1, 5$ and 10), so a bang-bang position is not a limitation of the tree here; it is what the problem asks for.
- **A higher fee does not make positions gradual:** they stay at 0 or 1 with a wider no-trade zone, as (6) says. Turnover falls (0.09 to 0.33 per decision) and so does everything else: Sharpe 0.52–0.61 against 0.57–0.88 for the base, and at edge 0.05 the tree stops trading at the fast rates.
- **Quartile candidates trade precision for stability.** At the fast rates the threshold is less precise (62% of the gain captured at every 2 days, against 83%): the candidates are quantiles of a persistent feature, and over the 11 fitting years the sample median of x is itself off by about ± 0.2 (autocorrelation time $1/k = 50$ days), whereas the full search reads the threshold

off the returns. At the slow rates, where the data cannot do better anyway, the coarse grid costs nothing (73% against 71% monthly) and its protection against small leaves shows: 20% of the monthly trees keep no split against 28%, and the thresholds' medians are the closest to d of all settings (+0.06 to +0.09).

9 A third experiment: quantile candidates, refinement and soft splits

Pipeline used. P3b: the four split-search options of `model_and_training_pipeline.tex` (Section 8 there), the deciles taken by rank among the admissible thresholds; a first run with the deciles of the values (P3) gave the same picture and was replaced by this one.

Why. Sections 4 and 8 showed two sources of error: the threshold moves by ± 0.1 to ± 0.3 from one dataset to the next, and a split on a handful of decisions occasionally survives pruning. Three options were added to the split search, all off by default (the training document `model_and_training_pipeline.tex`, Section 8, gives the exact rules):

- *Quantile candidates:* only nine candidates are screened, the deciles *by rank* of the admissible thresholds (those leaving at least the leaf minimum on each side), so they are evenly spread over what is allowed and never on its boundary; and every child keeps at least 10% of its node. (Quantiles of the node's *values* would differ only at the two ends: their outermost candidates sit on the smallest allowed leaves; see `model_and_training_pipeline.tex`, Section 8.)
- *Refinement:* every partition between the two deciles neighbouring the best one is screened as well; with plain splits the refined threshold wins only if it beats the decile by one standard error of the regret difference;
- *Soft splits (smoothing $\kappa = 1$):* every candidate threshold θ keeps a Boltzmann weight $\pi_\theta \propto e^{-z_\theta/\kappa}$, z_θ its regret excess over the best candidate in standard errors; a decision at x belongs to the left leaf with the weight of the thresholds at or above x , so the tree's score becomes a continuous function of x and thresholds the data cannot tell apart share the weight.

Each option was run alone and all three together, with the world and the protocol of Section 8 (rates 2, 5, 10 and 21 days, edges 0.05 and 0.10, 40 datasets per cell, $\lambda = 1$). Two measurements are new: the tree's score as a function of x (Figure 12) and the *spread* of a soft root split, the distance between the 25% and 75% quantiles of its weights.

What the tree says as a function of x (weekly decisions, edge 0.1, 40 datasets per panel)

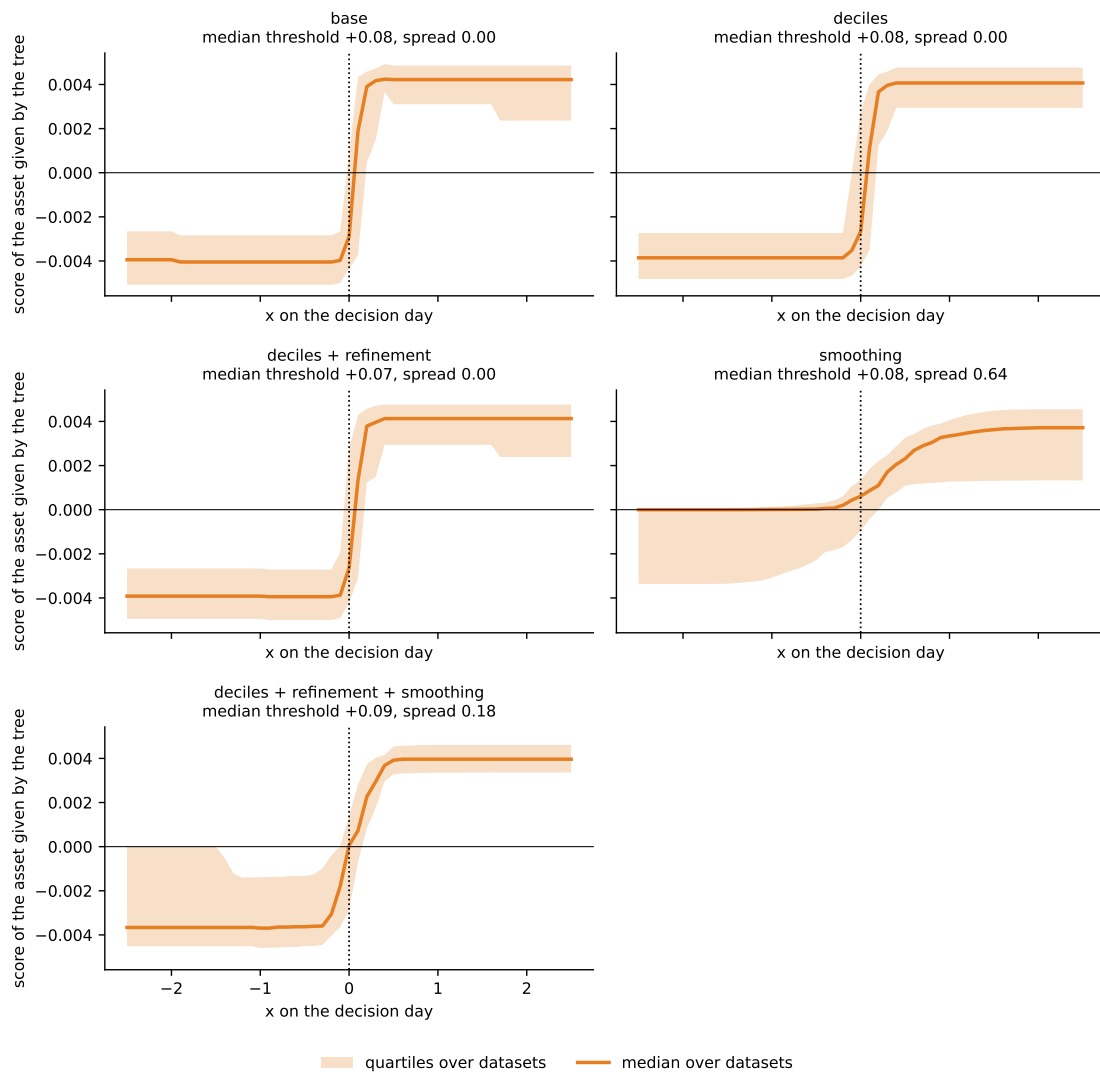


Figure 12: Weekly decisions, edge 0.10: the score the tree gives the asset as a function of x (median over the 40 datasets, quartiles as a band). Plain splits give a step; smoothing alone gives a ramp that stays near zero over half a standard deviation; the combination gives a step with a narrow soft edge.

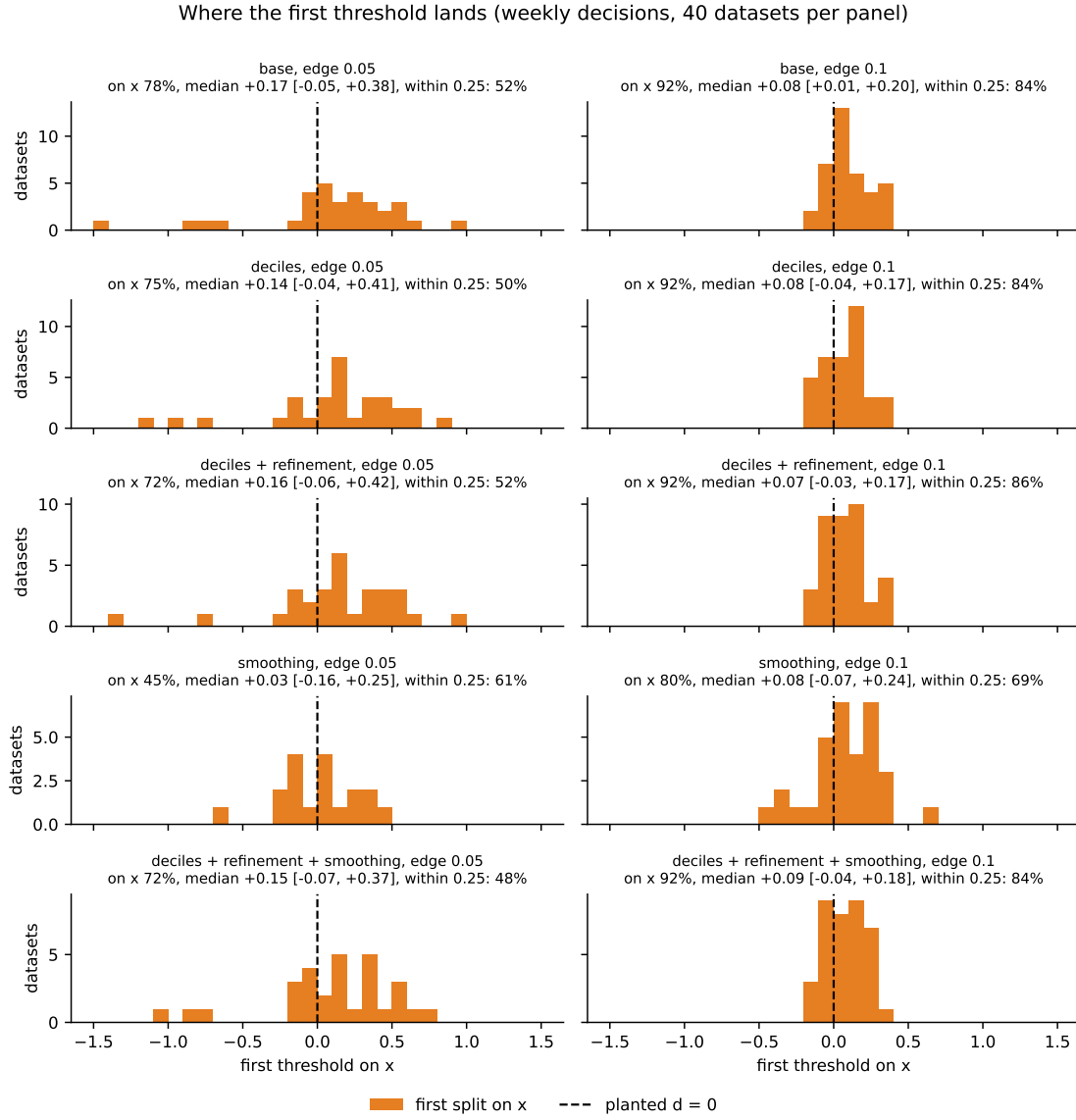


Figure 13: Weekly decisions: where the first threshold lands, one row per setting; *left*: edge 0.05, *right*: edge 0.10. The target is $d = 0$.

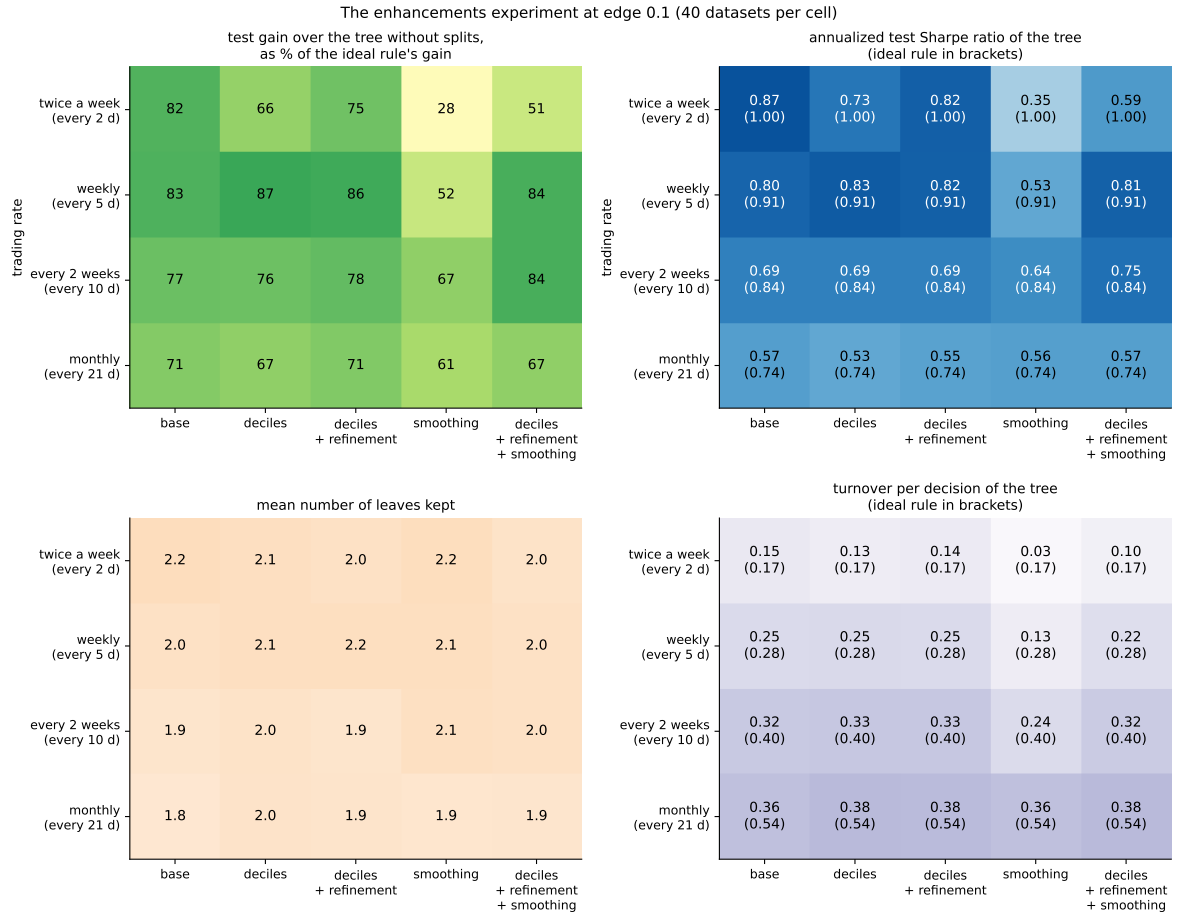


Figure 14: Edge 0.10, rows: trading rate, columns: setting. Share of the ideal rule's test gain captured; annualized test Sharpe ratio (ideal rule in brackets); mean number of leaves; turnover per decision (ideal rule in brackets).

setting	h	leaves	splits (%)		first threshold on x			annual. Sharpe		gain
			on x	none	median [quartiles]	$ \cdot \leq 0.25$	spread	tree	ideal	
base	2	2.2	98	2	+0.03 [−0.01, +0.12]	97%	–	0.87	1.00	82%
	5	2.0	92	8	+0.08 [+0.01, +0.20]	84%	–	0.80	0.91	83%
	10	1.9	88	12	+0.05 [−0.10, +0.26]	63%	–	0.69	0.84	77%
	21	1.8	72	28	+0.09 [−0.19, +0.29]	45%	–	0.57	0.74	71%
deciles	2	2.1	95	5	+0.06 [−0.03, +0.12]	87%	–	0.73	1.00	66%
	5	2.1	92	8	+0.08 [−0.04, +0.17]	84%	–	0.83	0.91	87%
	10	2.0	88	12	+0.11 [−0.03, +0.22]	71%	–	0.69	0.84	76%
	21	2.0	75	25	+0.04 [−0.22, +0.19]	57%	–	0.53	0.74	67%
deciles + refinement	2	2.1	95	5	+0.07 [−0.04, +0.12]	87%	–	0.82	1.00	75%
	5	2.2	92	8	+0.07 [−0.03, +0.17]	86%	–	0.82	0.91	86%
	10	1.9	88	12	+0.07 [−0.09, +0.24]	71%	–	0.69	0.84	78%
	21	2.0	75	25	+0.04 [−0.21, +0.20]	57%	–	0.55	0.74	71%
smoothing	2	2.2	78	22	+0.09 [−0.02, +0.23]	74%	0.65	0.35	1.00	28%
	5	2.2	80	20	+0.08 [−0.07, +0.24]	69%	0.64	0.53	0.91	52%
	10	2.1	80	20	+0.07 [−0.05, +0.23]	62%	0.70	0.64	0.84	67%
	21	1.9	75	25	+0.10 [−0.08, +0.17]	87%	0.68	0.56	0.74	61%
all three	2	2.0	85	15	+0.05 [+0.00, +0.12]	85%	0.18	0.59	1.00	51%
	5	2.1	92	8	+0.09 [−0.04, +0.18]	84%	0.18	0.81	0.91	84%
	10	2.1	92	8	+0.07 [−0.07, +0.27]	68%	0.19	0.75	0.84	84%
	21	1.9	80	20	+0.03 [−0.16, +0.18]	69%	0.21	0.57	0.74	67%

Table 7: The split-search options at edge 0.10, 40 datasets per row (h : days between decisions; turnover is in Figure 14). At edge 0.05 no setting improves on the base (gain captured 36–48% for the base, 28–56% with deciles, with or without refinement, 13–48% with all three), and smoothing alone loses most of it (8–23%).

Reading the figures and the table.

- **Deciles make the threshold steadier at the slow rates:** the share within 0.25 of d rises from 63 to 71% every two weeks and from 45 to 57% monthly, and the quartile width falls from 0.36 to 0.25 and from 0.48 to 0.41. Weekly and every two weeks the Sharpe ratio is unchanged; at the fastest rate the coarse grid costs 0.14 of Sharpe (0.73 against 0.87), because there the data can pin the threshold finer than a decile.
- **The refinement removes most of that cost** (0.82 at every 2 days, 75% of the gain against 66%) and keeps the steadier thresholds. Deciles with refinement match the base within noise everywhere (Sharpe 0.82, 0.82, 0.69, 0.55 against 0.87, 0.80, 0.69, 0.57) and centre the threshold better at the slow rates: a reasonable default for slow trading.
- **Smoothing alone, at $\kappa = 1$, is too wide and it interacts badly with the fee.** The weights spread over about ± 0.3 (spread 0.65): near d the expected return is small, so thresholds within that band are within one standard error of each other. The score then rises gradually from about zero (Figure 12) and, with $\lambda = 1$, stays inside the no-trade window of the optimizer for most decisions: the tree trades three to five times less (turnover 0.03 against 0.15 at every 2 days) and captures 28 to 67% of the gain instead of 77 to 83%. Smoothing the *score* is not sizing the *position*: with a proportional fee a small score means “do not trade”, not “trade a little”.
- **All three together:** the candidate set (the deciles and the refinement window) confines the weights, spread 0.18 to 0.21, and the threshold is as steady as with deciles (85, 84, 68 and 69% within 0.25). Every two weeks this setting captures the most of all (84%) with the best Sharpe ratio (0.75 against 0.69), monthly it matches the base (0.57), and at every 2 days it still pays for the soft edge (51% of the gain, Sharpe 0.59).
- **Next steps.** A sharper weight ($\kappa \approx 0.3$, or $e^{-z^2/2\kappa^2}$) should keep the soft edge where the data are undecided without pushing the score into the no-trade window; and smoothing is worth more where positions are meant to be interior, that is with $\lambda \geq 5$ (Section 8) or a quadratic

cost. Deciles with refinement can be switched on now for slow trading rates. The spurious second-level splits (10 to 25% of the datasets keep one, Table 7 counts them in the leaves) are the object of Section 10.

10 A fourth experiment: a calibrated pruning hurdle

Pipeline used. P3b with `prune_standard_errors` set to 1 or 2, alone and on top of deciles with refinement.

Why. Pruning keeps a split if, on the checking decisions, it lowers the mean regret by more than 10^{-6} and raises the Sharpe ratio by more than 0. Those hurdles are effectively zero, so a split confirmed by a handful of noisy decisions passes; that is how the spurious second-level leaves of Sections 4 and 9 survive. A new option, `prune_standard_errors =  $k_p$` , keeps a split only if its checking-row gain $\bar{g}$ (mean over the checking decisions of the regret without the split minus the regret with it) exceeds k_p standard errors of that mean, a paired test (`model_and_training_pipeline.tex`, Section 9, step 5). It was run with $k_p = 1$ and $k_p = 2$, alone and on top of deciles with refinement, in the world and protocol of Section 8 (rates 2, 5, 10 and 21 days, edges 0.05 and 0.10, 40 datasets per cell, $\lambda = 1$).

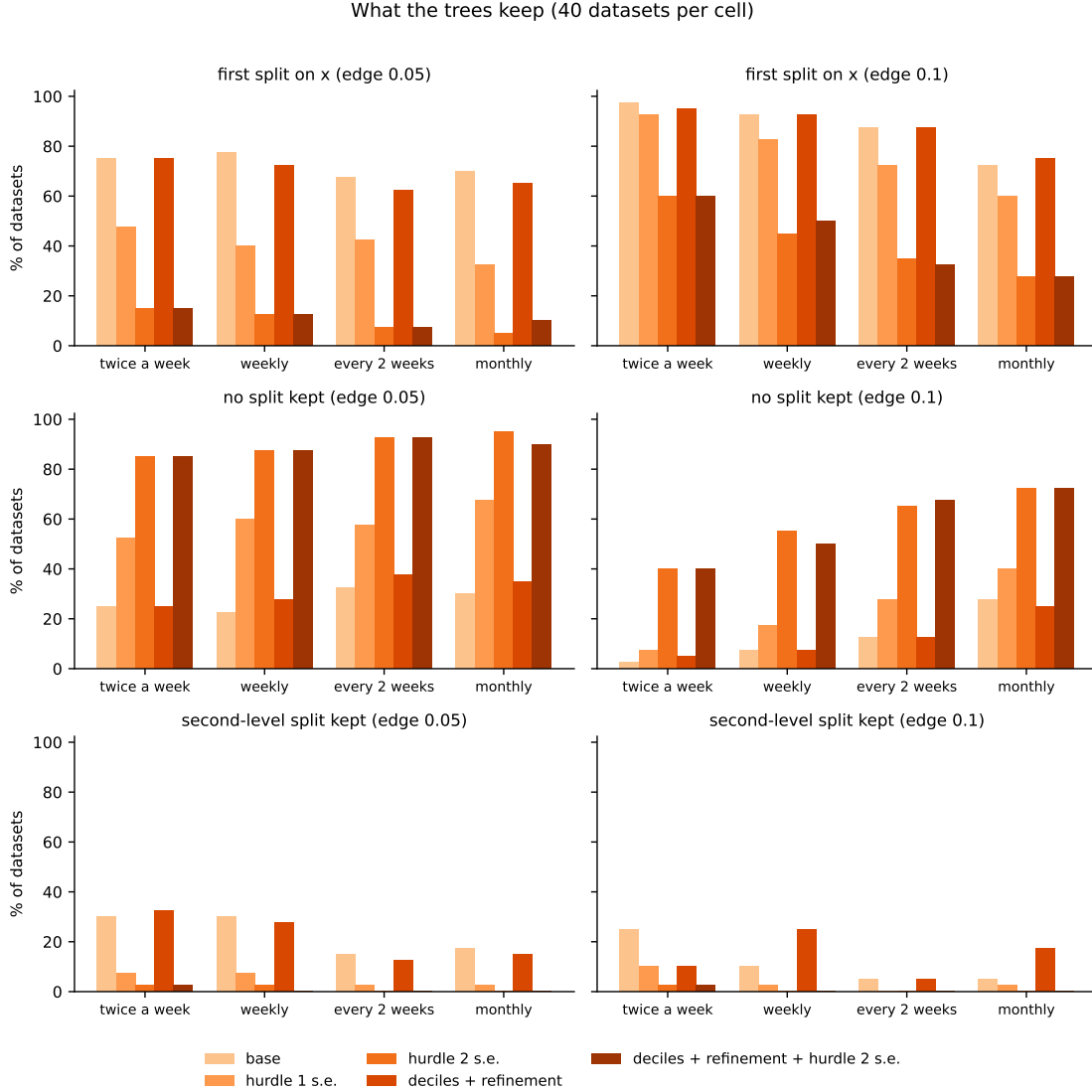


Figure 15: What the trees keep, by setting and trading rate: first split on x , no split kept, a second-level split kept (% of the datasets). *Top:* edge 0.05; *bottom:* edge 0.10.

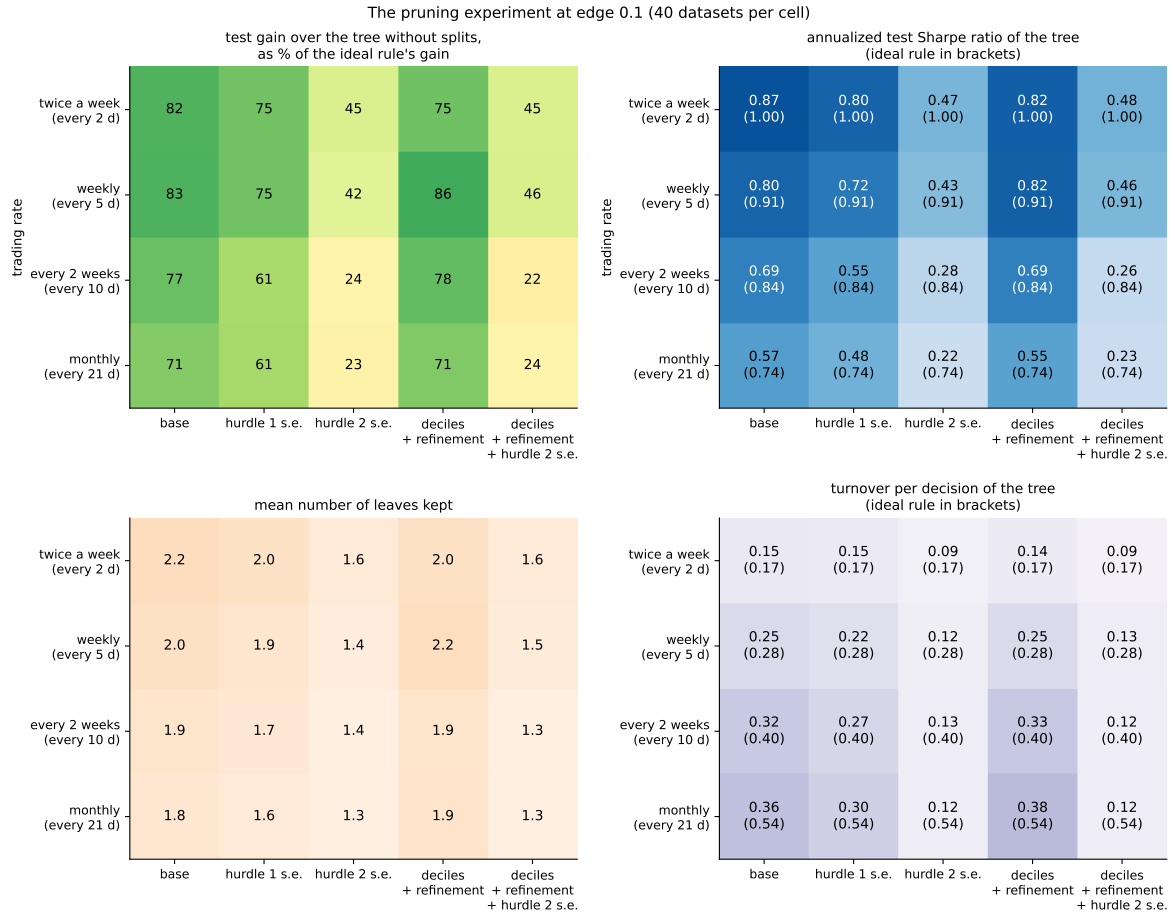


Figure 16: Edge 0.10, rows: trading rate, columns: setting. Share of the ideal rule's test gain captured; annualized test Sharpe ratio (ideal rule in brackets); mean number of leaves; turnover per decision (ideal rule in brackets).

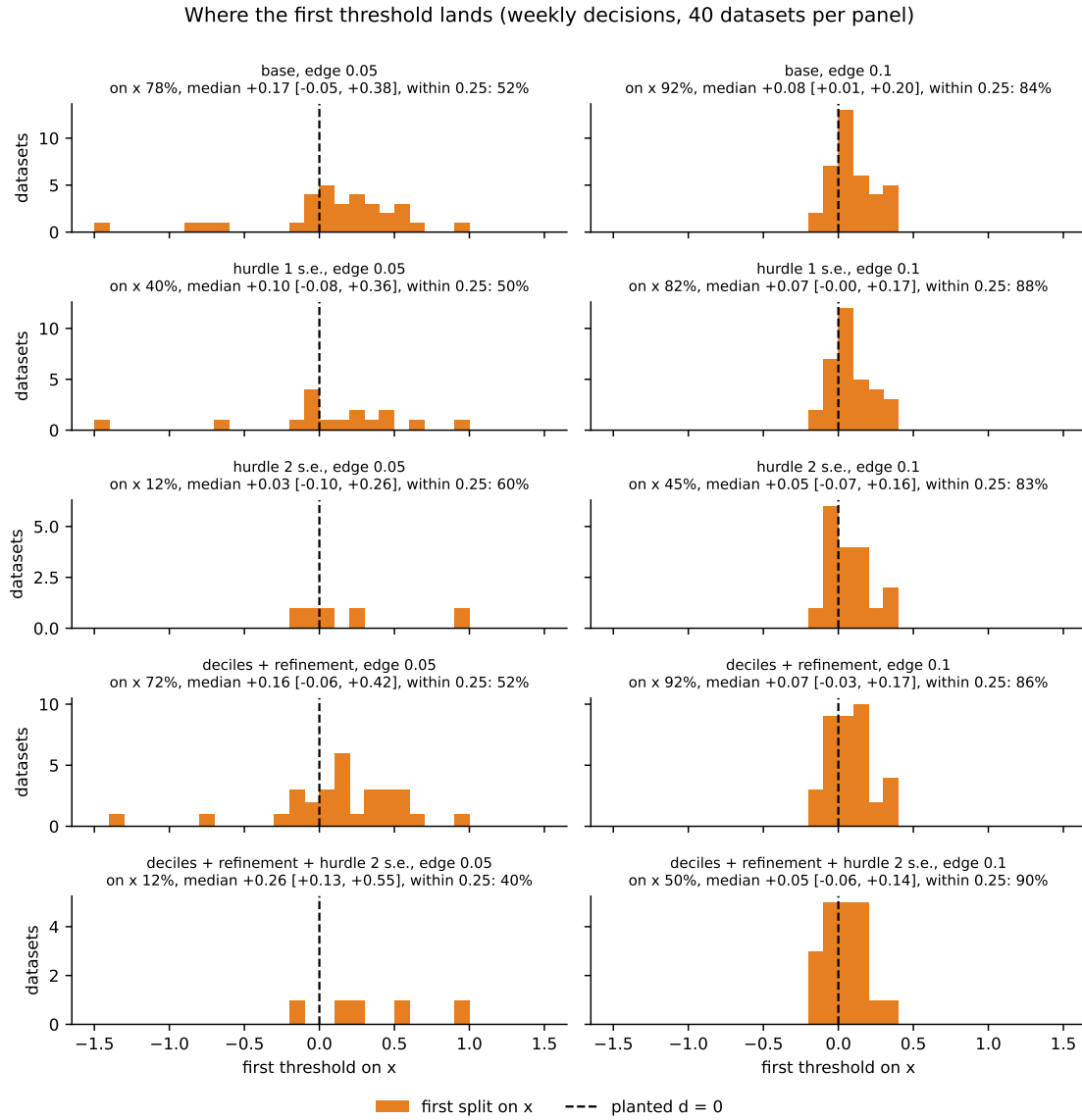


Figure 17: Weekly decisions: where the first threshold lands, one row per setting; *left*: edge 0.05, *right*: edge 0.10.

setting	h	datasets (%)			threshold $ \cdot \leq 0.25$	annual. Sharpe		gain captured
		on x	none	2nd level		tree	ideal	
base	2	98	2	25	97%	0.87	1.00	82%
	5	92	8	10	84%	0.80	0.91	83%
	10	88	12	5	63%	0.69	0.84	77%
	21	72	28	5	45%	0.57	0.74	71%
hurdle 1 s.e.	2	92	8	10	100%	0.80	1.00	75%
	5	82	18	2	88%	0.72	0.91	75%
	10	72	28	0	66%	0.55	0.84	61%
	21	60	40	2	50%	0.48	0.74	61%
hurdle 2 s.e.	2	60	40	2	100%	0.47	1.00	45%
	5	45	55	0	83%	0.43	0.91	42%
	10	35	65	0	43%	0.28	0.84	24%
	21	28	72	0	55%	0.22	0.74	23%
deciles + refinement	2	95	5	10	87%	0.82	1.00	75%
	5	92	8	25	86%	0.82	0.91	86%
	10	88	12	5	71%	0.69	0.84	78%
	21	75	25	18	57%	0.55	0.74	71%
deciles + refinement + hurdle 2 s.e.	2	60	40	2	92%	0.48	1.00	45%
	5	50	50	0	90%	0.46	0.91	46%
	10	32	68	0	62%	0.26	0.84	22%
	21	28	72	0	45%	0.23	0.74	24%

Table 8: The pruning hurdle at edge 0.10, 40 datasets per row (h : days between decisions; “2nd level”: datasets that keep a second-level split). At edge 0.05 the hurdle of 1 s.e. halves the trees that split (32–48% against 68–78%) and their Sharpe ratio (0.10–0.15 against 0.19–0.27); at 2 s.e. almost nothing survives (5–15% split, Sharpe about 0.05).

Reading the figures and the table.

- **The hurdle does remove the spurious leaves:** at 1 s.e. the second-level splits fall from 25, 10, 5 and 5% of the datasets to 10, 2, 0 and 2%, and at 2 s.e. to 2, 0, 0 and 0%; the thresholds that survive are the best-centred of all the experiments (100 and 88% within 0.25 at every 2 days and weekly).
- **But it removes true splits faster than it earns.** At 1 s.e. the first split on x falls from 98, 92, 88 and 72% to 92, 82, 72 and 60%, and the Sharpe ratio by 0.07 to 0.14; at 2 s.e. about half of the root splits are gone (45% at weekly decisions) and the Sharpe ratio halves. The tree without splits is a poor fallback here: it holds cash or drifts, so every true split lost costs the whole edge.
- **Why the test has so little power.** The checking block is a quarter of the training rows, 189 decisions at weekly trading. For dataset 42 the root split’s gain there is 0.0031 ± 0.0011 per decision ($z = 2.8$) and the tail leaf’s 0.00014 ± 0.0003 ($z = 0.5$): a true split at edge 0.10 typically clears one standard error and often not two, while the noise leaves sit below one. A hurdle that separates them needs more checking decisions than a 25% block provides, and at edge 0.05 no hurdle can separate them at all.
- **Deciles with refinement change nothing on this point:** the same hurdle gives the same losses on top of them.
- **What to do instead.** The spurious leaves are second-level splits, while the root split is confirmed 92 to 98% of the time at edge 0.10 without any hurdle. A hurdle applied only below the root (a depth-dependent k_p) would remove the noise leaves at no cost to the root split; a larger checking block (40% instead of 25%) would raise the power of the test at the cost of

growth data. Both are cheap to test. Until then, $k_p = 0$ stays the default, and the spurious leaves are better limited by the minimum leaf fraction of Section 9.

11 A fifth experiment: the resolution of the threshold against noise

Pipeline used. P3b with every option off, identical to P2; market M2 with observation noise on the feature (M2’).

Why. Sections 4 to 10 fixed the noise: a volatility of 1% a day and a feature observed exactly. Two questions remain about what the tree can resolve. How far can the price volatility rise, at a fixed drift, before the two regimes can no longer be told apart? And how much observation noise on the feature can the threshold survive? The second is not a question about the variance of x : the tree only sees the ordering of x , so scaling x changes nothing; what blurs a feature is noise *added* to it, $x_t^{\text{obs}} = x_t + \tau \varepsilon_t$ with ε_t white, the price still following the true x_t . Two sweeps, at weekly and monthly decisions, 60 datasets per point:

- the price volatility from 0.4% to 4% a day at a fixed regime drift of $\pm 25\%$ a year, so the edge $e = \mu/\sigma$ runs from 0.25 down to 0.025;
- the observation noise τ from 0 to 2 standard deviations of x , at the base volatility.

With a noisy feature the tree sees x^{obs} , and so does the ideal rule: it is fed with the expected return given the *observed* value, $\mathbb{E}[g_h(x) \mid x^{\text{obs}}]$, computed by drawing the true x from its Gaussian posterior (the true value shrinks towards the mean by $s^2/(s^2 + \tau^2)$). The threshold to find stays $d = 0$ by symmetry.

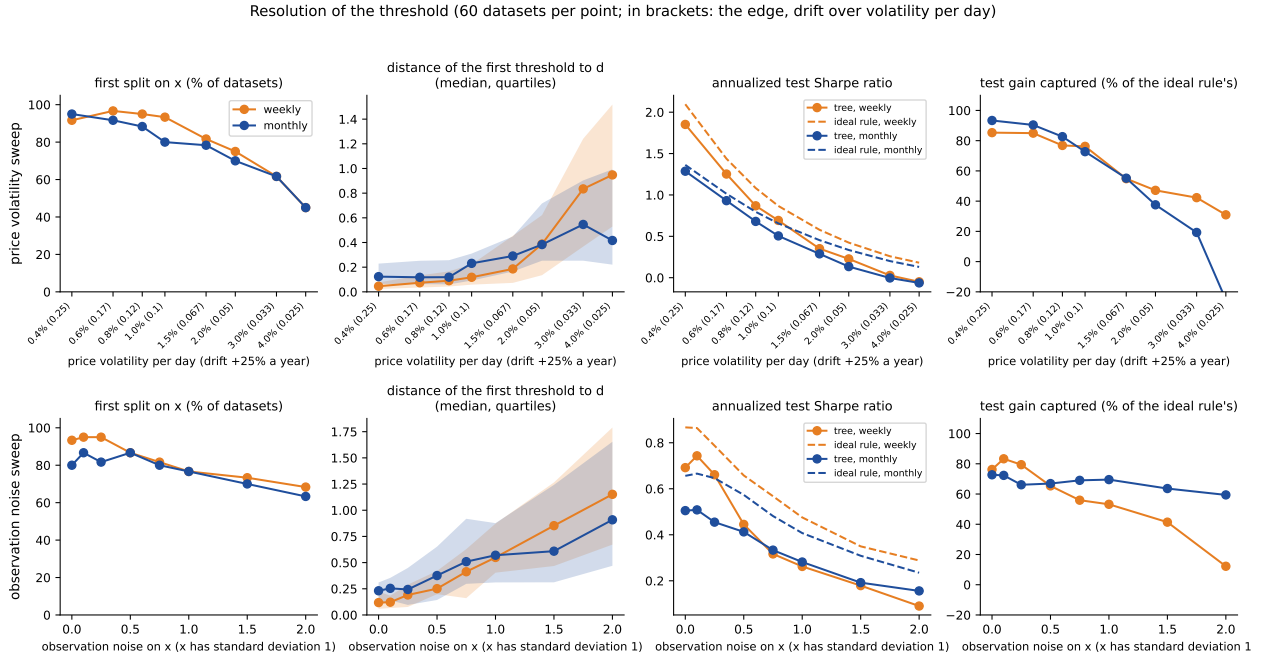


Figure 18: *Top:* the price volatility sweep (the edge in brackets); *bottom:* the observation-noise sweep. From left to right: datasets whose first split is on x ; distance of the first threshold to d (median and quartiles over the datasets that split); annualized test Sharpe ratio of the tree (solid) and of the ideal rule (dashed); share of the ideal rule’s gain captured. Orange: weekly decisions; blue: monthly.

sweep	value	edge	splits (%)		first threshold θ : median [quartiles]	annual. Sharpe		gain captured
			on x	none		tree	ideal	
volatility, weekly	0.4%	0.25	92	8	0.05 [0.02, 0.08]	1.85	2.10	85%
	0.6%	0.17	97	3	0.07 [0.03, 0.13]	1.25	1.44	85%
	0.8%	0.125	95	5	0.09 [0.04, 0.17]	0.87	1.08	77%
	1.0%	0.10	93	7	0.12 [0.06, 0.22]	0.69	0.87	76%
	1.5%	0.067	82	18	0.19 [0.07, 0.45]	0.35	0.58	55%
	2.0%	0.05	75	25	0.39 [0.14, 0.62]	0.23	0.42	47%
	3.0%	0.033	62	38	0.83 [0.37, 1.24]	0.03	0.26	42%
	4.0%	0.025	45	55	0.95 [0.53, 1.52]	-0.05	0.18	31%
volatility, monthly	0.4%	0.25	95	5	0.12 [0.06, 0.23]	1.29	1.36	93%
	0.6%	0.17	92	8	0.12 [0.06, 0.25]	0.93	1.01	90%
	0.8%	0.125	88	12	0.12 [0.06, 0.26]	0.68	0.80	83%
	1.0%	0.10	80	20	0.23 [0.09, 0.31]	0.51	0.66	73%
	1.5%	0.067	78	22	0.29 [0.16, 0.45]	0.29	0.45	55%
	2.0%	0.05	70	30	0.38 [0.25, 0.72]	0.13	0.33	38%
	3.0%	0.033	62	38	0.55 [0.25, 0.90]	0.00	0.20	19%
	4.0%	0.025	45	55	0.42 [0.22, 0.99]	-0.06	0.13	-25%
noise τ , weekly	0	0.10	93	7	0.12 [0.06, 0.22]	0.69	0.87	76%
	0.1	0.10	95	5	0.12 [0.07, 0.18]	0.74	0.86	83%
	0.25	0.10	95	5	0.19 [0.08, 0.29]	0.66	0.79	79%
	0.5	0.10	87	13	0.25 [0.20, 0.42]	0.45	0.66	65%
	0.75	0.10	82	18	0.41 [0.16, 0.63]	0.32	0.57	56%
	1	0.10	77	23	0.55 [0.40, 0.87]	0.26	0.48	53%
	1.5	0.10	73	27	0.85 [0.47, 1.27]	0.18	0.35	41%
	2	0.10	68	32	1.15 [0.67, 1.79]	0.09	0.29	12%
noise τ , monthly	0	0.10	80	20	0.23 [0.09, 0.31]	0.51	0.66	73%
	0.1	0.10	87	13	0.25 [0.14, 0.35]	0.51	0.67	72%
	0.25	0.10	82	18	0.24 [0.10, 0.45]	0.45	0.65	66%
	0.5	0.10	87	13	0.38 [0.14, 0.65]	0.41	0.57	67%
	0.75	0.10	80	20	0.51 [0.30, 0.92]	0.33	0.48	69%
	1	0.10	77	23	0.57 [0.31, 0.88]	0.28	0.41	70%
	1.5	0.10	70	30	0.61 [0.31, 1.24]	0.19	0.31	64%
	2	0.10	63	37	0.91 [0.47, 1.65]	0.16	0.24	59%

Table 9: The resolution sweeps, 60 datasets per row. The volatility sweep keeps the drift at $\pm 25\%$ a year, so the edge $e = \mu/\sigma$ falls as the volatility rises; the noise sweep keeps the base volatility and adds white noise of standard deviation τ to the observed feature (x has standard deviation 1). “Ideal” is the optimizer fed with the expected return given the observed feature; its own Sharpe ratio falls with the noise because the observed feature carries less information.

Reading the figure and the table.

- **The threshold error grows like one over the edge, then breaks.** At weekly decisions the median distance to d is 0.05, 0.09, 0.12 and 0.19 standard deviations of x for edges 0.25, 0.125, 0.10 and 0.067 (about $0.012/e$), then 0.39 at 0.05 and 0.83 at 0.033: below an edge of about 0.05 the split is no longer located, it is guessed. The share of datasets that split follows the same break (93% at 0.10, 75% at 0.05, 45% at 0.025), and so does the gain captured (76%, 47%, 31%).
- **Monthly decisions resolve less** (0.12 to 0.23 at the same edges, and 45% found at the largest volatility as well) but capture a comparable share of a smaller ideal gain; at a volatility of 4% a day the monthly tree earns less than no split at all.
- **Observation noise acts like a smaller edge.** Noise of τ standard deviations shrinks the information in the observed feature by $1/(1 + \tau^2)$, and the curves agree with that: $\tau = 1$ at edge 0.10 gives 77% found, an error of 0.55 and 53% of the gain, the same picture as an edge of 0.05 without noise (75%, 0.39, 47%). Up to $\tau = 0.25$ nothing is lost (error 0.19, 79% of the gain); at $\tau = 0.5$ the error doubles and the captured share falls to 65%; at $\tau = 2$ the tree

keeps 12% of what is left.

- **The ideal rule degrades too:** with a noisy feature the best possible Sharpe ratio falls from 0.87 to 0.29 at $\tau = 2$. The tree's loss relative to it (0.18 to 0.20 of Sharpe at every noise level up to $\tau = 1$) is roughly constant: noise removes information from everybody, and the tree's own estimation error stays what it was.
- **Practical reading.** For a regime feature to be worth a weekly tree, the drift-to-volatility ratio per day should be at least about 0.07 (a regime drift of $\pm 17\%$ a year at 1% daily volatility) and the feature should be observed with noise below a quarter of its own spread; a feature twice as noisy as that halves the attainable gain before the tree even starts.

12 How to reproduce

The code is in `code/` (run the commands below from that folder, with the project's Python, `../.venv/bin/python`); results and figures are in `report/figures/`. The results were produced with `synthetic_market.py` as committed in `eae145c` (one volatility for both regimes).

- `python planted_threshold_test.py`: one dataset, printed in detail (about 10 s).
- `python planted_threshold_experiments.py run`: the 2,000 fits in parallel, 103 minutes on 10 cores; writes `results_daily_threshold.json`.
- `python planted_threshold_experiments.py plot`: the figures and the summary table of Tables 2–3.
- `python -m unittest discover -s tests`: 59 tests, including the alignment and no-look-ahead checks of the rows and the Sharpe-of-cash convention.
- `python planted_threshold_experiments.py run-settings`: the settings experiment of Section 8, 1,920 fits, 54 minutes; writes `results_risk_fee_depth.json`, which `plot` then turns into Figures 10–11 and Table 6.
- `python planted_threshold_experiments.py run-enhancements`: the split-search options of Section 9, 1,600 fits, 35 minutes; writes `results_split_options.json`, drawn by `plot` as Figures 12–14 and Table 7.
- `python planted_threshold_experiments.py run-pruning`: the pruning hurdle of Section 10, 1,600 fits, 30 minutes; writes `results_pruning_hurdle.json`, drawn by `plot` as Figures 15–17 and Table 8.
- `python planted_threshold_experiments.py run-resolution`: the resolution sweeps of Section 11, 1,920 fits; writes `results_resolution.json`, drawn by `plot` as Figure 18.
- `report/model_and_training_pipeline.tex`: the model and its training, step by step, with the equations of every option.